

DETECCIÓN DE ANOMALÍAS Y TÉCNICAS DE AGRUPAMIENTO

Objetivo

Mediante este trabajo se pretende que poner en práctica la aplicación de los algoritmos de detección de anomalías u outliers y las técnicas de agrupamiento. El objetivo es detectar anomalías, generar y analizar agrupaciones aplicando algoritmos de aprendizaje máquina, con el fin de elegir el mejor método para el conjunto de datos dado.

En la presente actividad se va a emplear un dataset que contiene información de un conjunto de datos, que proviene de estudios de cardiotocografía (CTG) y contiene medidas extraídas de las señales de frecuencia cardíaca fetal (FHR) y contracciones uterinas.

Se procesaron automáticamente 2126 cardiotocogramas fetales (CTG) y se midieron las características diagnósticas respectivas. Los CTG también fueron clasificados por tres obstetras expertos y se asignó una etiqueta de clasificación consensuada a cada uno de ellos. La clasificación se realizó tanto con respecto a un patrón morfológico (A, B, C, ...) como al estado fetal (N, S, P). Por lo tanto, el conjunto de datos puede usarse tanto para experimentos de 10 clases como de 3 clases.

Después de un análisis se puede decir que, la información del *dataset* es:

INFORMACIÓN GENERAL

- *FileName / SegFile*: Nombre del archivo original y del segmento analizado.
- *Date*: Fecha en la que se realizó el estudio.
- *b / e*: Instante de inicio (b) y de fin (e) del segmento de la señal.

CARACTERÍSTICAS DE LA FRECUENCIA CARDÍACA FETAL (FHR)

- *LBE (Fetal Heart Rate Baseline)*: Es la línea de base de la frecuencia cardíaca fetal en latidos por minuto calculada de forma automática por el software SisPorto (Sistema informático utilizado para procesar los cardiotocogramas).
- *LB (Baseline Estimate)*: Es la estimación de la línea de base realizada por los expertos obstetras en latidos por minuto.
- *AC*: Número de aceleraciones por segundo.
- *FM*: Número de movimientos fetales por segundo.
- *UC*: Número de contracciones uterinas por segundo.
- *DL*: Número de desaceleraciones ligeras por segundo.
- *DS*: Número de desaceleraciones severas por segundo.

- *DP*: Número de desaceleraciones prolongadas por segundo.
- *DR*: Número de desaceleraciones repetitivas por segundo.

MEDIDAS DE VARIABILIDAD

- *ASTV*: Porcentaje de tiempo con variabilidad anormal a corto plazo.
- *MSTV*: Valor medio de la variabilidad a corto plazo.
- *ALTV*: Porcentaje de tiempo con variabilidad anormal a largo plazo.
- *MLTV*: Valor medio de la variabilidad a largo plazo.

ESTADÍSTICAS DEL HISTOGRAMA

Distribución de la Frecuencia Cardíaca Fetal (FHR)

- *Width*: Ancho del histograma de la frecuencia cardíaca.
- *Min / Max*: Valores mínimo y máximo del histograma.
- *Nmax*: Número de picos en el histograma.
- *Nzeros*: Número de ceros en el histograma.
- *Mode / Mean / Median*: Moda, media y mediana de la frecuencia cardíaca.
- *Variance*: Varianza del histograma.
- *Tendency*: Tendencia del histograma (-1: asimetría a la izquierda, 0: simétrico, 1: asimetría a la derecha).

CLASIFICACIÓN MORFOLÓGICA (Patrones de los Obstetras)

Estas columnas representan 10 clases basadas en el patrón visual (son valores binarios 0 o 1):

- *A*: Sueño tranquilo.
- *B*: Sueño REM.
- *C*: Vigilancia tranquila.
- *D*: Vigilancia activa.
- *E*: Patrón de cambio (A o B con oscilaciones).
- *AD*: Patrón acelerativo/desacelerativo (estrés).
- *DE*: Patrón desacelerativo (estrés).
- *LD*: Patrón de desaceleración prolongada.
- *FS*: Patrón plano-sinusoidal (patológico).

- *SUSP*: Patrón sospechoso.
- *CLASS*: Código numérico del patrón morfológico (del 1 al 10, correspondiente a las letras anteriores).

CLASIFICACIÓN DEL ESTADO FETAL

NSP: Esta es la etiqueta de clasificación de 3 clases (Estado fetal):

- 1 (*N*): Normal.
- 2 (*S*): Sospechoso (Suspect).
- 3 (*P*): Patológico.

IMPORTACIÓN DE LIBRERIAS A UTILIZAR

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sb

from sklearn.preprocessing import StandardScaler, RobustScaler,
MinMaxScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder

from sklearn import metrics
from sklearn import svm
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report, ConfusionMatrixDisplay

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, calinski_harabasz_score
from sklearn.metrics import homogeneity_score, completeness_score
from sklearn.metrics import v_measure_score, adjusted_rand_score
from sklearn.metrics import adjusted_mutual_info_score
import seaborn as sns

from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM
from math import pi
from sklearn.decomposition import PCA

from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram
```

[illegible]

```

NaN
2127      NaN      NaN      ...      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN
2128  564.0    23.0      ...      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN      NaN
NaN

```

```
[2129 rows x 40 columns]
```

#El tamaño del dataset: existen 21 columnas con 2000 datos cada una

```

print(f'El dataset tiene {df.shape[0]} filas (registros) y
{df.shape[1]} columnas (variables)')

```

El dataset tiene 2129 filas (registros) y 40 columnas (variables)

Se muestran las primeras 5 filas de datos

```
df.head(5)
```

	FileName	Date	SegFile	b	e	LBE	LB
AC \							
0	Variab10.txt	12/1/1996	CTG0001.txt	240.0	357.0	120.0	120.0
0.0							
1	Fmcs_1.txt	5/3/1996	CTG0002.txt	5.0	632.0	132.0	132.0
4.0							
2	Fmcs_1.txt	5/3/1996	CTG0003.txt	177.0	779.0	133.0	133.0
2.0							
3	Fmcs_1.txt	5/3/1996	CTG0004.txt	411.0	1192.0	134.0	134.0
2.0							
4	Fmcs_1.txt	5/3/1996	CTG0005.txt	533.0	1147.0	132.0	132.0
4.0							

	FM	UC	...	C	D	E	AD	DE	LD	FS	SUSP	CLASS	NSP
0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	9.0	2.0
1	0.0	4.0	...	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	6.0	1.0
2	0.0	5.0	...	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	6.0	1.0
3	0.0	6.0	...	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	6.0	1.0
4	0.0	5.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	2.0	1.0

```
[5 rows x 40 columns]
```

#Se muestran las ultimas 5 filas de datos

```
df.tail(5)
```

	FileName	Date	SegFile	b	e	LBE
LB AC \						
2124	S8001045.dsp	6/6/1998	CTG2127.txt	1576.0	3049.0	140.0

```

140.0  1.0
2125  S8001045.dsp  6/6/1998  CTG2128.txt  2796.0  3415.0  142.0
142.0  1.0
2126          NaN          NaN          NaN          NaN          NaN          NaN
NaN  NaN
2127          NaN          NaN          NaN          NaN          NaN          NaN
NaN  NaN
2128          NaN          NaN          NaN          NaN          NaN          NaN
NaN  NaN

          FM    UC    ...    C    D    E    AD    DE    LD    FS    SUSP    CLASS
NSP
2124    0.0    9.0    ...    0.0    0.0    1.0    0.0    0.0    0.0    0.0    0.0    5.0
2.0
2125    1.0    5.0    ...    0.0    0.0    0.0    0.0    0.0    0.0    0.0    0.0    1.0
1.0
2126    NaN    NaN    ...    NaN    NaN    NaN    NaN    NaN    NaN    NaN    NaN    NaN
NaN
2127    NaN    NaN    ...    NaN    NaN    NaN    NaN    NaN    NaN    NaN    NaN    NaN
NaN
2128  564.0   23.0    ...    NaN    NaN    NaN    NaN    NaN    NaN    NaN    NaN    NaN
NaN

[5 rows x 40 columns]

```

- Se observa que, las tres ultimas filas del dataset contienen datos faltantes o nulos.

1. ANÁLISIS DESCRIPTIVO DE DATOS

Limpieza de datos

```

# Por el metodo .info se encontraron 3 filas del dataset con datos
nulos
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2129 entries, 0 to 2128
Data columns (total 40 columns):
#   Column      Non-Null Count  Dtype
---  -
0   FileName    2126 non-null   object
1   Date        2126 non-null   object
2   SegFile     2126 non-null   object
3   b           2126 non-null   float64
4   e           2126 non-null   float64
5   LBE         2126 non-null   float64
6   LB          2126 non-null   float64
7   AC          2126 non-null   float64

```

8	FM	2127	non-null	float64
9	UC	2127	non-null	float64
10	ASTV	2127	non-null	float64
11	MSTV	2127	non-null	float64
12	ALTV	2127	non-null	float64
13	MLTV	2127	non-null	float64
14	DL	2128	non-null	float64
15	DS	2128	non-null	float64
16	DP	2128	non-null	float64
17	DR	2128	non-null	float64
18	Width	2126	non-null	float64
19	Min	2126	non-null	float64
20	Max	2126	non-null	float64
21	Nmax	2126	non-null	float64
22	Nzeros	2126	non-null	float64
23	Mode	2126	non-null	float64
24	Mean	2126	non-null	float64
25	Median	2126	non-null	float64
26	Variance	2126	non-null	float64
27	Tendency	2126	non-null	float64
28	A	2126	non-null	float64
29	B	2126	non-null	float64
30	C	2126	non-null	float64
31	D	2126	non-null	float64
32	E	2126	non-null	float64
33	AD	2126	non-null	float64
34	DE	2126	non-null	float64
35	LD	2126	non-null	float64
36	FS	2126	non-null	float64
37	SUSP	2126	non-null	float64
38	CLASS	2126	non-null	float64
39	NSP	2126	non-null	float64

dtypes: float64(37), object(3)
memory usage: 665.4+ KB

- Las variables FM, UC, ASTV, MSTV, ALTV, MLTV, DL, DS, DP, DR, apenas contienen algunos datos aislados en sus dos ultimas filas, pero no aportan valor al estudio, por lo que se ha decidido eliminar estas últimas 3 filas del conjunto de datos, para trabajar únicamente con los 2126 registros de datos completos.

Se eliminan las tres ultimas filas que contienen datos faltantes
df_3del = df.drop(df.index[-3:])

#Se muestran las ultimas 5 filas de datos
df_3del.tail(5)

	FileName	Date	SegFile	b	e	LBE
LB AC \						
2121	S8001045.dsp	6/6/1998	CTG2124.txt	2059.0	2867.0	140.0
140.0	0.0					

2122	S8001045.dsp	6/6/1998	CTG2125.txt	1576.0	2867.0	140.0
140.0	1.0					
2123	S8001045.dsp	6/6/1998	CTG2126.txt	1576.0	2596.0	140.0
140.0	1.0					
2124	S8001045.dsp	6/6/1998	CTG2127.txt	1576.0	3049.0	140.0
140.0	1.0					
2125	S8001045.dsp	6/6/1998	CTG2128.txt	2796.0	3415.0	142.0
142.0	1.0					

	FM	UC	...	C	D	E	AD	DE	LD	FS	SUSP	CLASS
NSP												
2121	0.0	6.0	...	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	5.0
2.0												
2122	0.0	9.0	...	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	5.0
2.0												
2123	0.0	7.0	...	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	5.0
2.0												
2124	0.0	9.0	...	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	5.0
2.0												
2125	1.0	5.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0
1.0												

[5 rows x 40 columns]

#El tamaño del dataset: existen 21 columnas con 2000 datos cada una

```
print(f'El dataset tiene {df_3del.shape[0]} filas (registros) y {df_3del.shape[1]} columnas (variables)')
```

El dataset tiene 2126 filas (registros) y 40 columnas (variables)

- Se han eliminado las tres ultimas filas con datos faltantes, no contienen información que aporte valor al estudio.

Se calcula cuantos valores NaN existen en el nuevo dataset

```
df_3del.isna().sum()
```

FileName	0
Date	0
SegFile	0
b	0
e	0
LBE	0
LB	0
AC	0
FM	0
UC	0
ASTV	0
MSTV	0
ALTV	0

```
MLTV      0
DL         0
DS         0
DP         0
DR         0
Width     0
Min        0
Max        0
Nmax       0
Nzeros     0
Mode       0
Mean       0
Median     0
Variance   0
Tendency   0
A          0
B          0
C          0
D          0
E          0
AD         0
DE         0
LD         0
FS         0
SUSP       0
CLASS      0
NSP        0
dtype: int64
```

Por el metodo .info no se encontraron datos nulos en el dataset
df_3del.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2126 entries, 0 to 2125
Data columns (total 40 columns):
#   Column      Non-Null Count  Dtype
---  -
0   FileName    2126 non-null  object
1   Date        2126 non-null  object
2   SegFile     2126 non-null  object
3   b           2126 non-null  float64
4   e           2126 non-null  float64
5   LBE         2126 non-null  float64
6   LB          2126 non-null  float64
7   AC          2126 non-null  float64
8   FM          2126 non-null  float64
9   UC          2126 non-null  float64
10  ASTV        2126 non-null  float64
11  MSTV        2126 non-null  float64
12  ALTV        2126 non-null  float64
```

13	MLTV	2126	non-null	float64
14	DL	2126	non-null	float64
15	DS	2126	non-null	float64
16	DP	2126	non-null	float64
17	DR	2126	non-null	float64
18	Width	2126	non-null	float64
19	Min	2126	non-null	float64
20	Max	2126	non-null	float64
21	Nmax	2126	non-null	float64
22	Nzeros	2126	non-null	float64
23	Mode	2126	non-null	float64
24	Mean	2126	non-null	float64
25	Median	2126	non-null	float64
26	Variance	2126	non-null	float64
27	Tendency	2126	non-null	float64
28	A	2126	non-null	float64
29	B	2126	non-null	float64
30	C	2126	non-null	float64
31	D	2126	non-null	float64
32	E	2126	non-null	float64
33	AD	2126	non-null	float64
34	DE	2126	non-null	float64
35	LD	2126	non-null	float64
36	FS	2126	non-null	float64
37	SUSP	2126	non-null	float64
38	CLASS	2126	non-null	float64
39	NSP	2126	non-null	float64

dtypes: float64(37), object(3)
memory usage: 664.5+ KB

- No se encontraron datos NaN en el nuevo dataset.

Variables

```
# Variables categóricas
lst_catvar = list(df_3del.select_dtypes(include='object'))
lst_catvar = lst_catvar + ['CLASS'] + ['NSP'] + ['Tendency']
print(f'Variables categóricas\n{lst_catvar}')

# Variables categóricas dicotómicas
lst_dicvar = [dic for dic in df_3del.columns if
df_3del[dic].nunique(dropna=True) == 2 and dic != 'DS']
#lst_dicvar = lst_dicvar
print(f'Variables categóricas dicotómicas\n{lst_dicvar}')

# Variables numéricas discretas
lst_disvar = list(df_3del.select_dtypes(include='int'))
lst_disvar = lst_disvar + ['AC'] + ['FM'] + ['UC'] + ['DL'] + ['DP']
+ ['Nmax'] + ['Nzeros'] + ['b'] + ['e'] + ['DR'] + ['DS']
print(f'Variables numéricas discretas\n{lst_disvar}')

# Variables numéricas continuas
```

```

lst_contvar = list( set(df_3del.columns) - set(lst_dicvar) -
set(lst_disvar) - set(lst_catvar))
print(f'Variables numéricas continuas\n{lst_contvar}')

Variables categóricas
['FileName', 'Date', 'SegFile', 'CLASS', 'NSP', 'Tendency']
Variables categóricas dicotómicas
['A', 'B', 'C', 'D', 'E', 'AD', 'DE', 'LD', 'FS', 'SUSP']
Variables numéricas discretas
['AC', 'FM', 'UC', 'DL', 'DP', 'Nmax', 'Nzeros', 'b', 'e', 'DR', 'DS']
Variables numéricas continuas
['Width', 'MSTV', 'Mode', 'ASTV', 'MLTV', 'Variance', 'LBE', 'LB',
'Median', 'Mean', 'Max', 'Min', 'ALTV']

# Variables categóricas dicotómicas
print('Variables categóricas dicotómicas\n')
for col in df_3del[lst_dicvar]:
    lst_unique = df_3del[col].unique()
    if len(lst_unique)<3:
        print(f'{col}: {lst_unique}')
    else:
        print(f'La variable {col} tiene {len(lst_unique)} valores
distintos')

Variables categóricas dicotómicas

A: [0. 1.]
B: [0. 1.]
C: [0. 1.]
D: [0. 1.]
E: [0. 1.]
AD: [0. 1.]
DE: [0. 1.]
LD: [0. 1.]
FS: [1. 0.]
SUSP: [0. 1.]

```

Variables Categóricas

Se ha identificado las variables categóricas que representan etiquetas o grupos, aunque en el dataset algunas de ellas han sido convertidas a números, su significado es cualitativo, estas son:

- CLASS: Código de la clasificación morfológica (del 1 al 10, donde 1=A, 2=B, etc.), podría ser considerada con variable objetivo para un caso de clasificación de 10 clases.
- NSP: Estado fetal (1 = Normal, 2 = Sospechoso, 3 = Patológico), podría ser considerada con variable objetivo para un caso de clasificación de 3 clases.
- Tendency: Dirección del histograma de la FHR (-1 = Asimétrico izquierda, 0 = Simétrico, 1 = Asimétrico derecha).

- FileName, Date, SegFile: Metadatos identificadores (textos y fechas).

Variables Categóricas Dicotómicas

Se han identificado las variables binarias que solo pueden tomar dos valores (0 y 1), indican ausencia o presencia de una característica y son:

- Patrones Morfológicos: A, B, C, D, E, AD, DE, LD, FS, SUSP.

```
# Variables numéricas discretas
print('Variables numéricas discretas\n')
for col in df_3del[lst_disvar]:
    lst_unique = df_3del[col].unique()
    if len(lst_unique)<1:
        print(f'{col}: {lst_unique}')
    else:
        print(f'La variable {col} tiene {len(lst_unique)} valores distintos')
```

Variables numéricas discretas

```
La variable AC tiene 22 valores distintos
La variable FM tiene 96 valores distintos
La variable UC tiene 19 valores distintos
La variable DL tiene 15 valores distintos
La variable DP tiene 5 valores distintos
La variable Nmax tiene 18 valores distintos
La variable Nzeros tiene 9 valores distintos
La variable b tiene 979 valores distintos
La variable e tiene 1064 valores distintos
La variable DR tiene 1 valores distintos
La variable DS tiene 2 valores distintos
```

```
# Variables numéricas continuas
print('Variables numéricas continuas\n')
for col in df_3del[lst_contvar]:
    lst_unique = df_3del[col].unique()
    if len(lst_unique)<1:
        print(f'{col}: {lst_unique}')
    else:
        print(f'La variable {col} tiene {len(lst_unique)} valores distintos')
```

Variables numéricas continuas

```
La variable Width tiene 154 valores distintos
La variable MSTV tiene 57 valores distintos
La variable Mode tiene 88 valores distintos
La variable ASTV tiene 75 valores distintos
La variable MLTV tiene 249 valores distintos
La variable Variance tiene 133 valores distintos
La variable LBE tiene 48 valores distintos
```

La variable LB tiene 48 valores distintos
La variable Median tiene 95 valores distintos
La variable Mean tiene 103 valores distintos
La variable Max tiene 86 valores distintos
La variable Min tiene 109 valores distintos
La variable ALTV tiene 87 valores distintos

Variables Numéricas Discretas

Se han identificado las variables numéricas discretas, sus valores representan conteos enteros o índices específicos y no admiten valores intermedios en su medición original, estas son:

- Eventos de conteo: AC (Aceleraciones), FM (Movimientos fetales), UC (Contracciones), DL (Desaceleraciones ligeras), DP (Desaceleraciones prolongadas), DS (Desaceleraciones severas), DR (Desaceleraciones repetitivas).
- Histograma: Nmax (Número de picos), Nzeros (Número de ceros).
- Límites: b y e (Índices de inicio y fin del segmento).

Variables Numéricas Continuas

Se ha identificado las variables numéricas continuas, pueden tomar cualquier valor numérico y son:

- Frecuencia Cardíaca: LBE y LB (Línea de base en lpm), Mean (Media), Median (Mediana), Mode (Moda).
- Variabilidad: ASTV y ALTV: Porcentajes de variabilidad anormal (se presentan como enteros pero representan una proporción continua).
- MSTV y MLTV: Valores medios de variabilidad (presentan decimales).
- Distribución: Width (Ancho del histograma), Min (Mínimo), Max (Máximo), Variance (Varianza).

Datos faltantes

Se analiza las columnas que tienen valores cero en sus datos, se excluyen las variables categóricas dicotómicas

```
cols_zeros = [c for c in df_3del.columns if c not in ['DS', 'A', 'B', 'C', 'D', 'E', 'AD', 'DE', 'LD', 'FS', 'SUSP', 'DR']]  
(df_3del[cols_zeros] == 0).sum(axis=0)
```

FileName	0
Date	0
SegFile	0
b	314
e	0
LBE	0

LB	0
AC	891
FM	1311
UC	332
ASTV	0
MSTV	0
ALTV	1240
MLTV	137
DL	1231
DP	1948
Width	0
Min	0
Max	0
Nmax	107
Nzeros	1624
Mode	0
Mean	0
Median	0
Variance	187
Tendency	1115
CLASS	0
NSP	0

dtype: int64

Algunas variables del dataset poseen datos con valor de cero, dependiendo de la variable esto puede tener un significado u otro desde el punto médico, se realiza un análisis para cada una de ellas, cabe mencionar que se ha dejado fuera a las variables categóricas dicotómicas por la naturaleza de sus datos.

Índice o límite del segmento

- b (límite del segmento) = 0: Puede significar que el análisis se realizó desde el momento cero (inicio) del segmento.

Eventos de Bienestar (AC, FM)

- AC (Aceleraciones) = 0: Puede significar que el corazón del bebé no aumentó su ritmo durante ese periodo. Si el bebé está durmiendo (sueño tranquilo), es normal. Sin embargo, si no hay aceleraciones durante mucho tiempo incluso cuando el bebé se mueve, puede indicar que el bebé no tiene suficientes reservas de oxígeno.
- FM (Movimientos Fetales) = 0: Significa que el bebé estuvo quieto durante el registro. Al igual que las aceleraciones, esto es normal durante los ciclos de sueño del feto (que duran unos 20-40 minutos).

Actividad Uterina (UC)

- UC (Contracciones) = 0: Significa que la madre no tuvo contracciones durante el monitoreo. Esto es lo esperado si la paciente no ha entrado en labor de parto o si se trata de un control de rutina.

Medidas de Variabilidad (ASTV, MSTV, ALTV, MLTV)

- ASTV y ALTV (Porcentaje de tiempo anormal) = 0: Significa que no se detectó una variabilidad fuera de los rangos normales. Es decir, el ritmo cardíaco se mantuvo "sano" todo el tiempo.
- MSTV y MLTV (Valor medio de variabilidad) = 0: Significa que el corazón late como un metrónomo exacto, sin ninguna fluctuación (línea plana o "silente"). En medicina fetal, esto es un signo de alarma grave, ya que un corazón sano siempre tiene pequeñas variaciones latido a latido.

Eventos de Estrés o Alerta (DL, DS, DP, DR)

- DL (Desaceleraciones ligeras) = 0: No hubo caídas leves en el ritmo cardíaco.
- DS (Desaceleraciones severas) = 0: No hubo caídas bruscas y peligrosas.
- DP (Desaceleraciones prolongadas) = 0: No hubo caídas que duraran más de 2 minutos.
- DR (Desaceleraciones repetitivas) = 0: No hubo un patrón cíclico de caídas.

Un cero en estas variables suele indicar un feto que no está bajo estrés mecánico o falta de oxígeno por compresiones del cordón.

CONCLUSIÓN

Luego de este análisis realizado con ayuda de la parte médica, se puede decir que no existen datos faltantes en el dataset, como se describió en los enunciados anteriores, un cero presente en estas variables tiene distintos significados desde el punto de vista médico.

BIBLIOGRAFÍA

Ayres-de-Campos, D., Spong, C. Y., & Chandrachan, E. (2015). FIGO consensus guidelines on intrapartum fetal monitoring: Cardiotocography. International Journal of Gynecology & Obstetrics, 131(1), 13-24.

Valores duplicados

```
df_3del.duplicated().sum()  
np.int64(0)
```

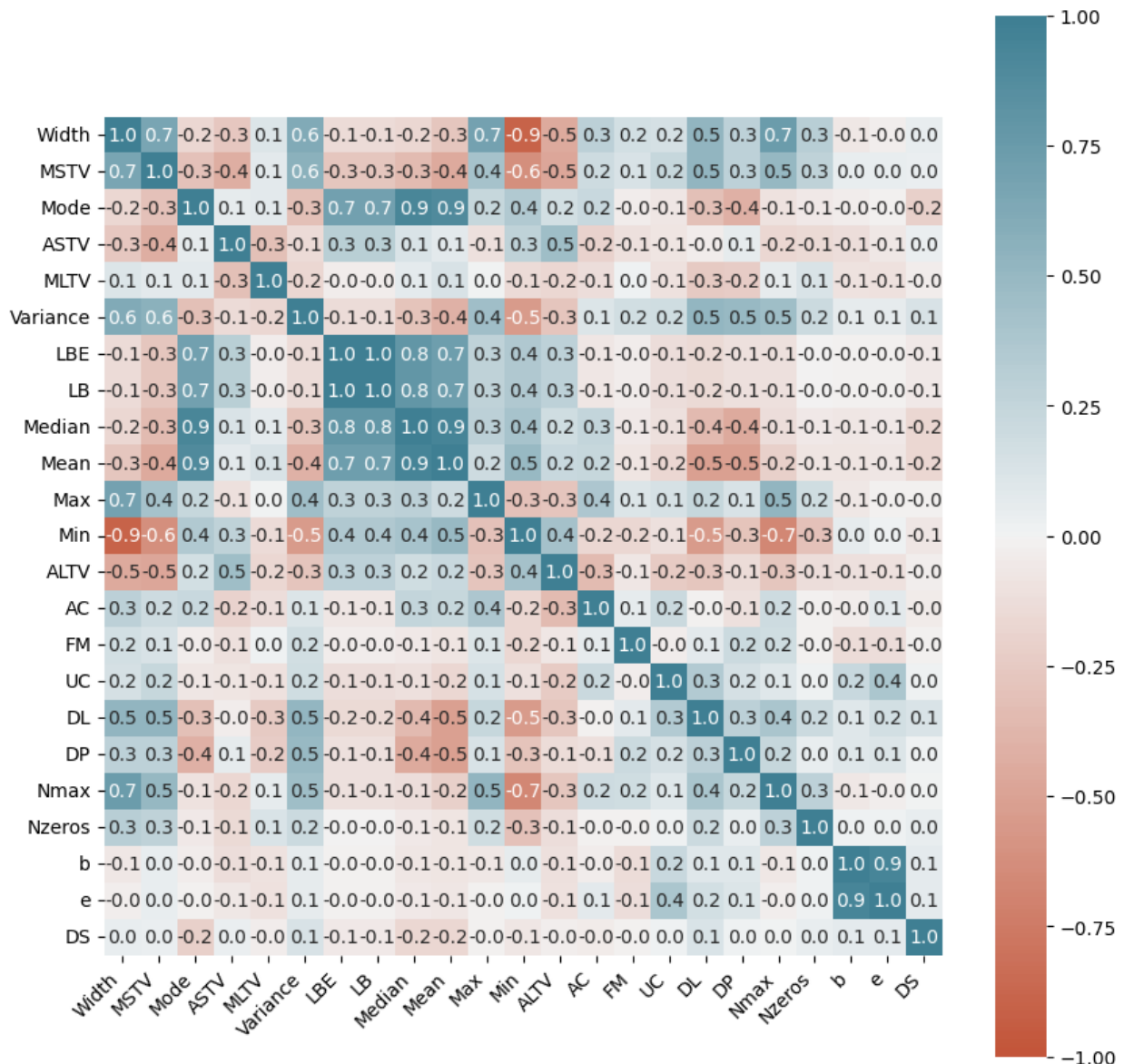
- No se encontraron datos duplicados en el dataset.

Variables numéricas

```
# Matriz decorrelaciones numericas original, se excluye la variable DR  
por no contener datos
```

```
lst_disvar.remove('DR')  
df_num = df_3del[lst_contvar+lst_disvar]
```

```
corr= df_num.corr()
plt.figure(figsize=(10, 10))
ax = sb.heatmap(
    corr,
    vmin=-1, vmax=1, center=0,
    cmap=sb.diverging_palette(20, 220, n=200),
    square=True,
    annot = True, fmt=".1f"
)
ax.set_xticklabels(
    ax.get_xticklabels(),
    rotation=45,
    horizontalalignment='right'
);
```



RELACIONES CLÍNICAS

- Las variables *LBE* (*Fetal Heart Rate Baseline*) y *LB* (*Baseline Estimate*), poseen una *correlación*=1, esto sugiere que poseen la misma información, es decir tanto el software SisPorto (el sistema informático utilizado para procesar los cardiotocogramas), como la estimación de la línea de base realizada por los expertos obstetras, es la misma.
- Las variables *Mean*, *Median* y *Mode*, poseen una *correlación*=0.9, esto sugiere que la distribución de los latidos del corazón de los fetos es bastante simétrica con ausencia de sesgo significativo, puede significar también que, estas medidas poseen valores muy cercanos entre si aportan información muy parecida.

- Las variables *LBE (Fetal Heart Rate Baseline)*, *LB (Baseline Estimate)* y *Median, Mode, Mean*, poseen una *correlación*=0.8, 0,7 y 0,7 respectivamente, sugiere que, la línea de base de la frecuencia cardíaca fetal en latidos por minuto es sobre la cual se calcularon los valores promedios con los datos de estas variables.

HISTOGRAMA

- Las variables *Width (Ancho del histograma de la FHR)* y *Min (Valor mínimo del histograma de la frecuencia cardíaca)*, poseen una fuerte *correlación negativa*= - 0.9, esto puede significar que el ancho del histograma está determinado principalmente por qué tan bajo cae la frecuencia cardíaca (las desaceleraciones), más que por qué tan alto sube.
- Las variables *Width (Ancho del histograma de la FHR)* y *NMax (Número de picos en el histograma)*, poseen una *correlación*=0.7, esto significa que cuanto más amplio es el rango de pulsaciones, más "picos" o modas locales aparecen en el gráfico, desde el punto clínico, esto tiene sentido, pues un feto activo pasa por diferentes estados de ritmo cardíaco.
- Las variables *NMax (Número de picos en el histograma)* y *Min (Valor mínimo del histograma)*, poseen una *correlación negativa*= - 0.7, esto significa que a menor valor de Min (frecuencia cardíaca más baja detectada), mayor tiende a ser el valor de Nmax (más picos en el histograma), es decir que son inversamente proporcionales entre si.
- La variable *Width (Ancho del histograma de la FHR)* y *MSTV (Valor medio de la variabilidad a corto plazo)*, *Variance (Varianza del histograma)* poseen una *correlación*=0,7 y 0,6 respectivamente, significa una relación importante entre el ancho del histograma y la varianza, que están ligados a la variabilidad a corto plazo. Si un feto tiene mucha variabilidad en su FHR, el histograma naturalmente se ensancha.
- Las variables *MSTV (Valor medio de la variabilidad a corto plazo)* y *Min (Valor mínimo del histograma)*, poseen una *correlación negativa*= - 0.6, significa que una variabilidad alta a corto plazo suele estar asociada a la presencia de caídas en el ritmo (desaceleraciones), lo que baja el valor mínimo registrado.

DATOS AISLADOS

- Las variables *b y e (Instante de inicio (b) y de fin (e) del segmento de la señal)*, poseen una *correlación*=0.9, solo entre ellas, lo que sugiere que no aportan ninguna relación especial con el resto de variables.
- Las variables *FM (Movimientos Fetales)* y *UC (Contracciones)*, muestran *correlaciones*<0.9 muy bajas en relación al resto de variables, es decir que estas variables aportan información independiente que no puede ser explicada por la frecuencia cardíaca.

Resumen estadístico de las variables numéricas

```
df_3del[lst_contvar+lst_disvar].describe().transpose()
```

	count	mean	std	min	25%	50%
75% \						
Width	2126.0	70.445908	38.955693	3.0	37.0	67.5
100.00						
MSTV	2126.0	1.332785	0.883241	0.2	0.7	1.2
1.70						
Mode	2126.0	137.452023	16.381289	60.0	129.0	139.0
148.00						
ASTV	2126.0	46.990122	17.192814	12.0	32.0	49.0
61.00						
MLTV	2126.0	8.187629	5.628247	0.0	4.6	7.4
10.80						
Variance	2126.0	18.808090	28.977636	0.0	2.0	7.0
24.00						
LBE	2126.0	133.303857	9.840844	106.0	126.0	133.0
140.00						
LB	2126.0	133.303857	9.840844	106.0	126.0	133.0
140.00						
Median	2126.0	138.090310	14.466589	77.0	129.0	139.0
148.00						
Mean	2126.0	134.610536	15.593596	73.0	125.0	136.0
145.00						
Max	2126.0	164.025400	17.944183	122.0	152.0	162.0
174.00						
Min	2126.0	93.579492	29.560212	50.0	67.0	93.0
120.00						
ALTV	2126.0	9.846660	18.396880	0.0	0.0	0.0
11.00						
AC	2126.0	2.722484	3.560850	0.0	0.0	1.0
4.00						
FM	2126.0	7.241298	37.125309	0.0	0.0	0.0
2.00						
UC	2126.0	3.659925	2.847094	0.0	1.0	3.0
5.00						
DL	2126.0	1.570085	2.499229	0.0	0.0	0.0
3.00						
DP	2126.0	0.126058	0.464361	0.0	0.0	0.0
0.00						
Nmax	2126.0	4.068203	2.949386	0.0	2.0	3.0
6.00						
Nzeros	2126.0	0.323612	0.706059	0.0	0.0	0.0
0.00						
b	2126.0	878.439793	894.084748	0.0	55.0	538.0
1521.00						
e	2126.0	1702.877234	930.919143	287.0	1009.0	1241.0
2434.75						

DS	2126.0	0.003293	0.057300	0.0	0.0	0.0
0.00						
	max					
Width	180.0					
MSTV	7.0					
Mode	187.0					
ASTV	87.0					
MLTV	50.7					
Variance	269.0					
LBE	160.0					
LB	160.0					
Median	186.0					
Mean	182.0					
Max	238.0					
Min	159.0					
ALTV	91.0					
AC	26.0					
FM	564.0					
UC	23.0					
DL	16.0					
DP	4.0					
Nmax	18.0					
Nzeros	10.0					
b	3296.0					
e	3599.0					
DS	1.0					

- Las variables *LBE (Fetal Heart Rate Baseline)* y *LB (Baseline Estimate)*, presentan datos estadísticos iguales, lo que confirma que poseen la misma información.
- Las variables *LBE (Fetal Heart Rate Baseline)*, *LB (Baseline Estimate)* y *Median*, *Mode*, *Mean*, presentan una distribución de datos muy similar, poseen una *media* = 133.3/133.3, 134.61, 138.09 y 137.45 respectivamente, esto significa que la gran mayoría de los registros corresponden a fetos con una frecuencia cardíaca normal y estable. El hecho de que la Moda y la Mediana sean casi iguales (139 lpm) sugiere que la distribución principal es unimodal y muy simétrica en el centro.
- Las variables *Nzeros (Número de ceros en el histograma)*, *DL (Número de desaceleraciones ligeras por segundo)*, *DP (Número de desaceleraciones prolongadas por segundo)*, *FM (Número de movimientos fetales por segundo)* y *ALTV (Porcentaje de tiempo con variabilidad anormal a largo plazo)*, poseen una concentración de sus datos en los últimos cuartiles, por consecuencia sus primeros cuartiles están vacíos y como estas variables representan valores anormales en la FHR y la mayoría de sus datos son cero, esto sugiere que basado en la distribución de estas variables y el análisis de los cuartiles reflejan que, la mayoría de los pacientes del dataset son fetos con FHR normal y que el conjunto de datos está fuertemente sesgado hacia estos fetos con estado de salud normal.

```
# Se realiza una copia del dataset para modificaciones
df_data = df_3del.copy()
```

```
# Se eliminan las variables no necesarias para el análisis de
anomalías y clusters.
df_data.drop(['FileName', 'Date', 'SegFile', 'LB', 'DR', 'b', 'e'],
axis=1, inplace=True)
df_data.head(10)
```

	LBE	AC	FM	UC	ASTV	MSTV	ALTV	MLTV	DL	DS	...	C
0	120.0	0.0	0.0	0.0	73.0	0.5	43.0	2.4	0.0	0.0	...	0.0
1	132.0	4.0	0.0	4.0	17.0	2.1	0.0	10.4	2.0	0.0	...	0.0
2	133.0	2.0	0.0	5.0	16.0	2.1	0.0	13.4	2.0	0.0	...	0.0
3	134.0	2.0	0.0	6.0	16.0	2.4	0.0	23.0	2.0	0.0	...	0.0
4	132.0	4.0	0.0	5.0	16.0	2.4	0.0	19.9	0.0	0.0	...	0.0
5	134.0	1.0	0.0	10.0	26.0	5.9	0.0	0.0	9.0	0.0	...	0.0
6	134.0	1.0	0.0	9.0	29.0	6.3	0.0	0.0	6.0	0.0	...	0.0
7	122.0	0.0	0.0	0.0	83.0	0.5	6.0	15.6	0.0	0.0	...	0.0
8	122.0	0.0	0.0	1.0	84.0	0.5	5.0	13.6	0.0	0.0	...	0.0
9	122.0	0.0	0.0	3.0	86.0	0.3	6.0	10.6	0.0	0.0	...	0.0

	E	AD	DE	LD	FS	SUSP	CLASS	NSP
0	0.0	0.0	0.0	0.0	1.0	0.0	9.0	2.0
1	0.0	1.0	0.0	0.0	0.0	0.0	6.0	1.0
2	0.0	1.0	0.0	0.0	0.0	0.0	6.0	1.0
3	0.0	1.0	0.0	0.0	0.0	0.0	6.0	1.0
4	0.0	0.0	0.0	0.0	0.0	0.0	2.0	1.0
5	0.0	0.0	0.0	1.0	0.0	0.0	8.0	3.0
6	0.0	0.0	0.0	1.0	0.0	0.0	8.0	3.0
7	0.0	0.0	0.0	0.0	1.0	0.0	9.0	3.0
8	0.0	0.0	0.0	0.0	1.0	0.0	9.0	3.0
9	0.0	0.0	0.0	0.0	1.0	0.0	9.0	3.0

[10 rows x 33 columns]

Se han eliminado estas variables por no contener información que aporte valor al estudio de anomalías y a la agrupación por clusters, estas son:

- FileName / SegFile: Nombre del archivo original y del segmento analizado.
- Date: Fecha en la que se realizó el estudio.
- b / e: Instante de inicio (b) y de fin (e) del segmento de la señal FHR analizada.

- LB (Baseline Estimate): Es la estimación de la línea de base realizada por los expertos obstetras en latidos por minuto, se ha eliminado por que contiene la misma informacion que su variable gemela LBE.
- DR: Número de desaceleraciones repetitivas por segundo, esta variable no contiene datos, todos sus registros son cero.

Tipos de Variables

```
# Variables categóricas
lst_catvar = list(df_data.select_dtypes(include='object'))
lst_catvar = lst_catvar + ['CLASS'] + ['NSP'] + ['Tendency']
print(f'Variables categóricas\n{lst_catvar}')

# Variables categóricas dicotómicas
lst_dicvar = [dic for dic in df_data.columns if
df_data[dic].nunique(dropna=True) == 2 and dic != 'DS']
lst_dicvar = lst_dicvar
print(f'Variables categóricas dicotómicas\n{lst_dicvar}')

# Variables numéricas discretas
lst_disvar = list(df_data.select_dtypes(include='int'))
lst_disvar = lst_disvar + ['AC'] + ['FM'] + ['UC'] + ['DL'] + ['DP']
+ ['Nmax'] + ['Nzeros'] + ['DS']
print(f'Variables numéricas discretas\n{lst_disvar}')

# Variables numéricas continuas
lst_contvar = list( set(df_data.columns) - set(lst_dicvar) -
set(lst_disvar) - set(lst_catvar))
print(f'Variables numéricas continuas\n{lst_contvar}')
```

Variables categóricas
['CLASS', 'NSP', 'Tendency']

Variables categóricas dicotómicas
['A', 'B', 'C', 'D', 'E', 'AD', 'DE', 'LD', 'FS', 'SUSP']

Variables numéricas discretas
['AC', 'FM', 'UC', 'DL', 'DP', 'Nmax', 'Nzeros', 'DS']

Variables numéricas continuas
['Width', 'MSTV', 'Mode', 'MLTV', 'Variance', 'LBE', 'Median', 'ASTV',
'Mean', 'Max', 'Min', 'ALTV']

Variables Categóricas

Variable Target NSP

```
# Diccionario de etiquetas
class_labels_1 = {1: 'Normal', 2: 'Sospechoso', 3: 'Patológico'}

plt.figure(figsize=(10, 4))

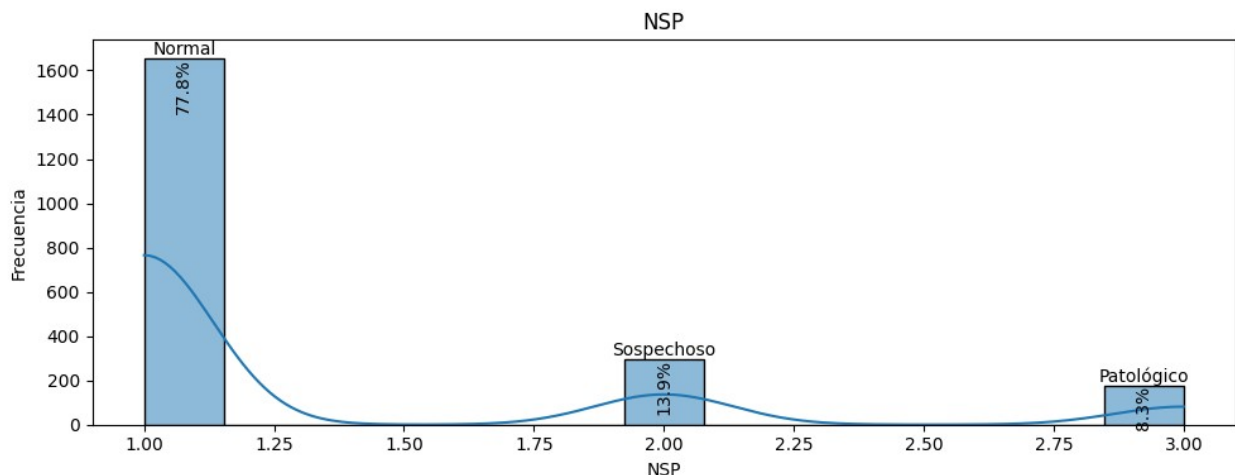
# Crear el histograma
ax = sb.histplot(df_data['NSP'], kde=True)
```

```

# Agregar porcentajes sobre cada barra con rotación
total = len(df_data[lst_catvar])
for p in ax.patches:
    height = p.get_height()
    if height > 0: # Solo añadir el porcentaje si la altura es mayor
que cero
        # Calcular el porcentaje
        percentage = f'{100 * height / total:.1f}%'
        x = p.get_x() + p.get_width() / 2
        y = p.get_y() + height
        value = int(round(x)) # redondear al valor de clase
        label = class_labels_1.get(value, '')
        # Añadir el porcentaje como anotación con rotación
        ax.annotate(percentage, (x, y), ha='center', va='top',
rotation=90)
        ax.text(x, y, label, ha='center', va='bottom', fontsize=10)

# Ajustes del gráfico
plt.title('NSP')
plt.xlabel('NSP')
plt.ylabel('Frecuencia')
plt.tight_layout()
plt.show()

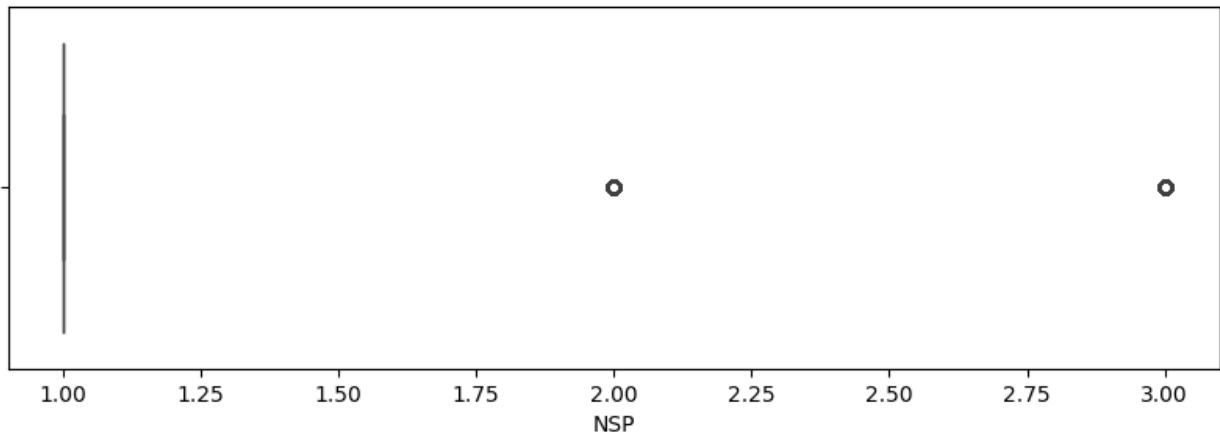
```



```

plt.figure(figsize=(10, 3))
sb.boxplot(df_data['NSP'], orient='h')
plt.show()

```



El dataset se encuentra balanceado en su variable de salida price_range

```
df_data['NSP'].value_counts()
```

```
NSP
1.0    1655
2.0     295
3.0     176
Name: count, dtype: int64
```

- El dataset se encuentra desbalanceado en su variable de salida NSP.

Variable Target CLASS

```
# Diccionario de etiquetas
class_labels_2 = {1: 'A', 2: 'B', 3: 'C', 4: 'D', 5: 'E', 6: 'AD', 7: 'DE', 8: 'LD', 9: 'FS', 10: 'SUSP'}

plt.figure(figsize=(10, 5))

# Crear el histograma
ax = sb.histplot(df_data['CLASS'], kde=True)

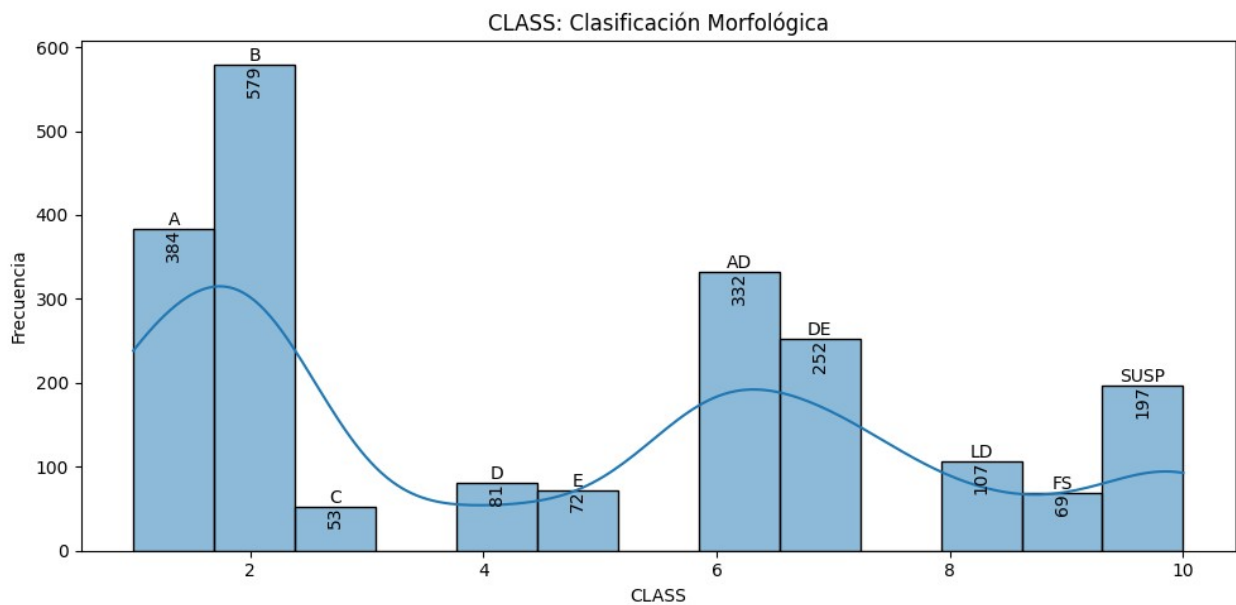
# Agregar porcentajes sobre cada barra con rotación
# total = len(df_data[lst_catvar])
for p in ax.patches:
    height = p.get_height()
    if height > 0: # Solo añadir el porcentaje si la altura es mayor que cero
        # Calcular el porcentaje
        # percentage = f'{100 * height / total:.1f}%'
        x = p.get_x() + p.get_width() / 2
        y = p.get_y() + height
        value = int(round(x)) # redondear al valor de clase
```

```

label = class_labels_2.get(value, '')
# Añadir el porcentaje como anotación con rotación
ax.annotate(height, (x, y), ha='center', va='top',
rotation=90)
ax.text(x, y, label, ha='center', va='bottom', fontsize=10)

# Ajustes del gráfico
plt.title('CLASS: Clasificación Morfológica')
plt.xlabel('CLASS')
plt.ylabel('Frecuencia')
plt.tight_layout()
plt.show()

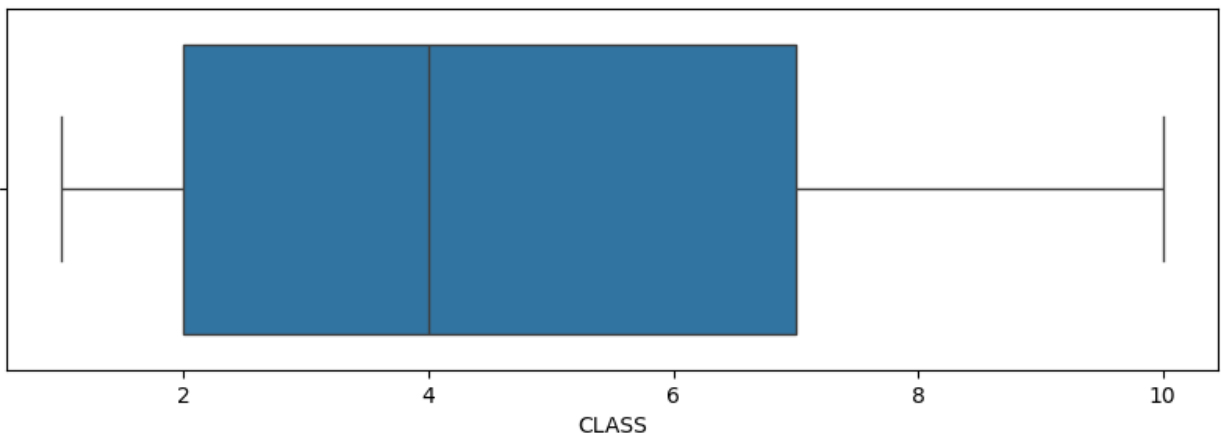
```



```

plt.figure(figsize=(10, 3))
sb.boxplot(df_data['CLASS'], orient='h')
plt.show()

```



```
# El dataset se encuentra balaceado en su variable de salida price_range
```

```
df_data['CLASS'].value_counts()
```

```
CLASS
2.0    579
1.0    384
6.0    332
7.0    252
10.0   197
8.0    107
4.0     81
5.0     72
9.0     69
3.0     53
Name: count, dtype: int64
```

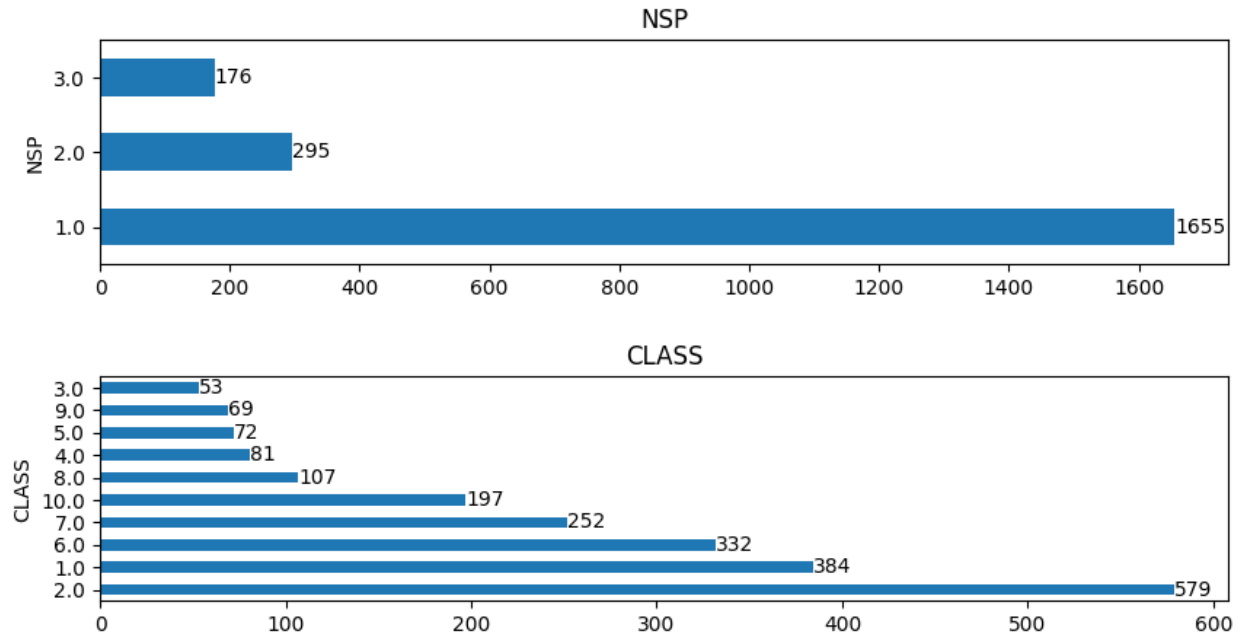
- El dataset se encuentra desbalaceado en su segunda variable de salida CLASS.

```
lst_num_1 = ['NSP', 'CLASS']

fig, axes = plt.subplots(nrows=2, ncols=1, figsize=(10,5))

plt.subplots_adjust(
    wspace=0.5,
    hspace=0.5
)

for col, ax in zip(df_data[lst_num_1], axes.ravel()):
    df_3del[col].value_counts().plot.barh(ax=ax)
    ax.set_title(col)
    ax.bar_label(ax.containers[0])
plt.show()
```



Variable Tendency

```
# Diccionario de etiquetas
class_labels_3 = {-1: 'Asimetrica - ', 0: 'Simétrica', 1: 'Asimetria + '
'}

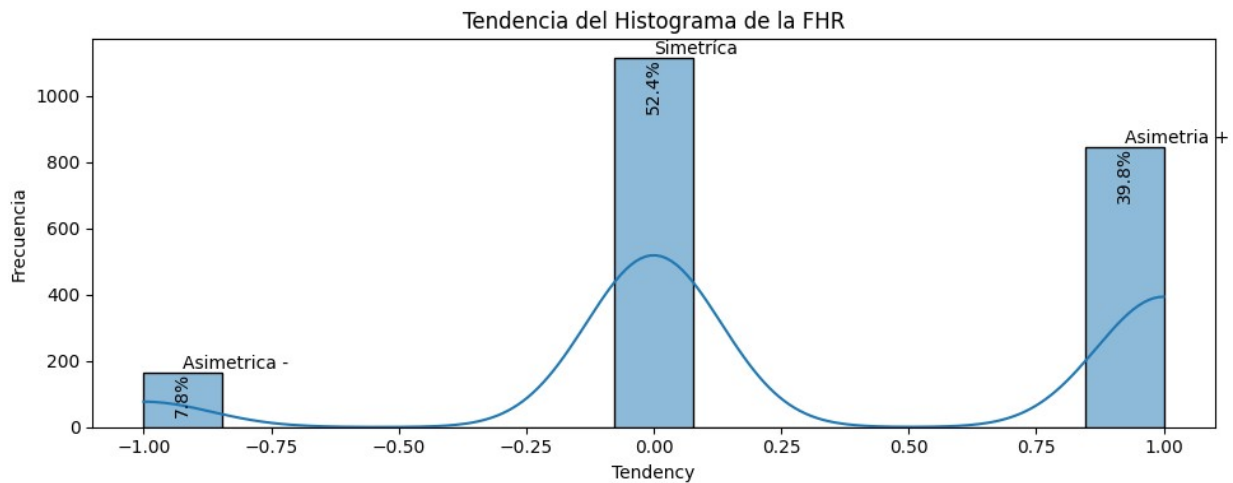
plt.figure(figsize=(10, 4))

# Crear el histograma
ax = sb.histplot(df_data['Tendency'], kde=True)

# Agregar porcentajes sobre cada barra con rotación
total = len(df_data[lst_catvar])
for p in ax.patches:
    height = p.get_height()
    if height > 0: # Solo añadir el porcentaje si la altura es mayor
que cero
        # Calcular el porcentaje
        percentage = f'{100 * height / total:.1f}%'
        x = p.get_x() + p.get_width() / 2
        y = p.get_y() + height
        value = int(round(x)) # redondear al valor de clase
        label = class_labels_3.get(value, '')
        # Añadir el porcentaje como anotación con rotación
        ax.annotate(percentaje, (x, y), ha='center', va='top',
rotation=90)
        ax.text(x, y, label, ha='left', va='bottom', fontsize=10)

# Ajustes del gráfico
```

```
plt.title('Tendencia del Histograma de la FHR')
plt.xlabel('Tendency')
plt.ylabel('Frecuencia')
plt.tight_layout()
plt.show()
```

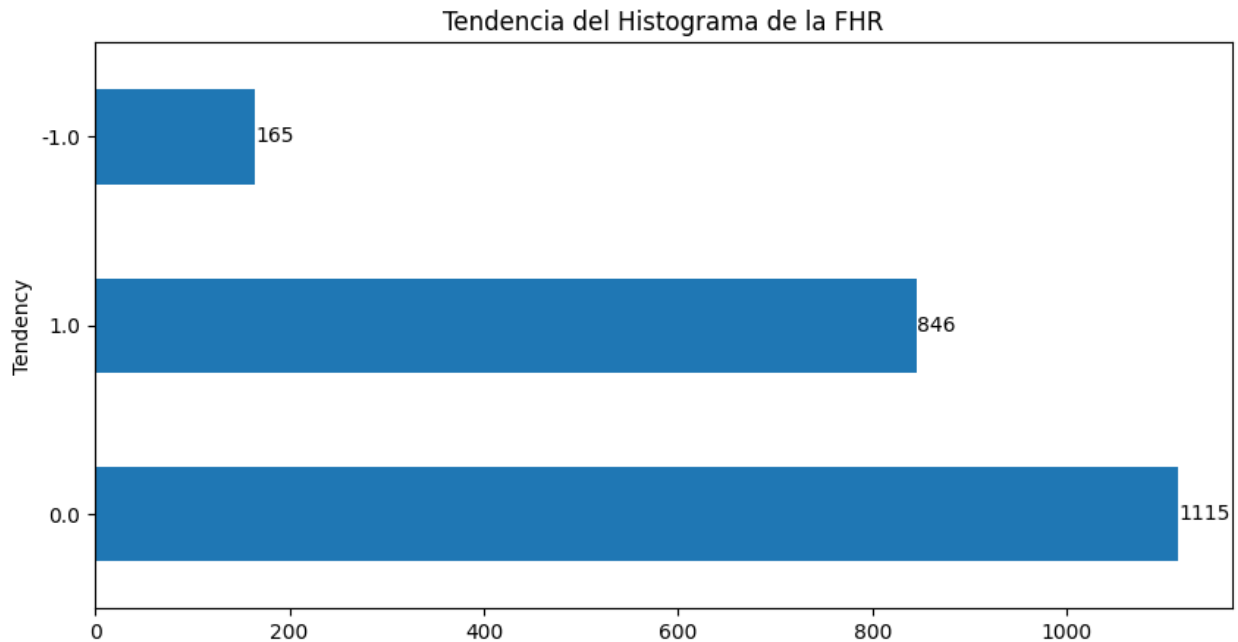


```
# Crear una sola figura y eje
fig, ax = plt.subplots(figsize=(10, 5))

# Graficar
df_data['Tendency'].value_counts().plot.barh(ax=ax)

# Títulos y etiquetas
ax.set_title(col)
ax.bar_label(ax.containers[0])

#plt.tight_layout()
plt.title('Tendencia del Histograma de la FHR')
plt.show()
```



Variable Categóricas Dicotómicas

Se grafican 4 variables categóricas dicotómicas

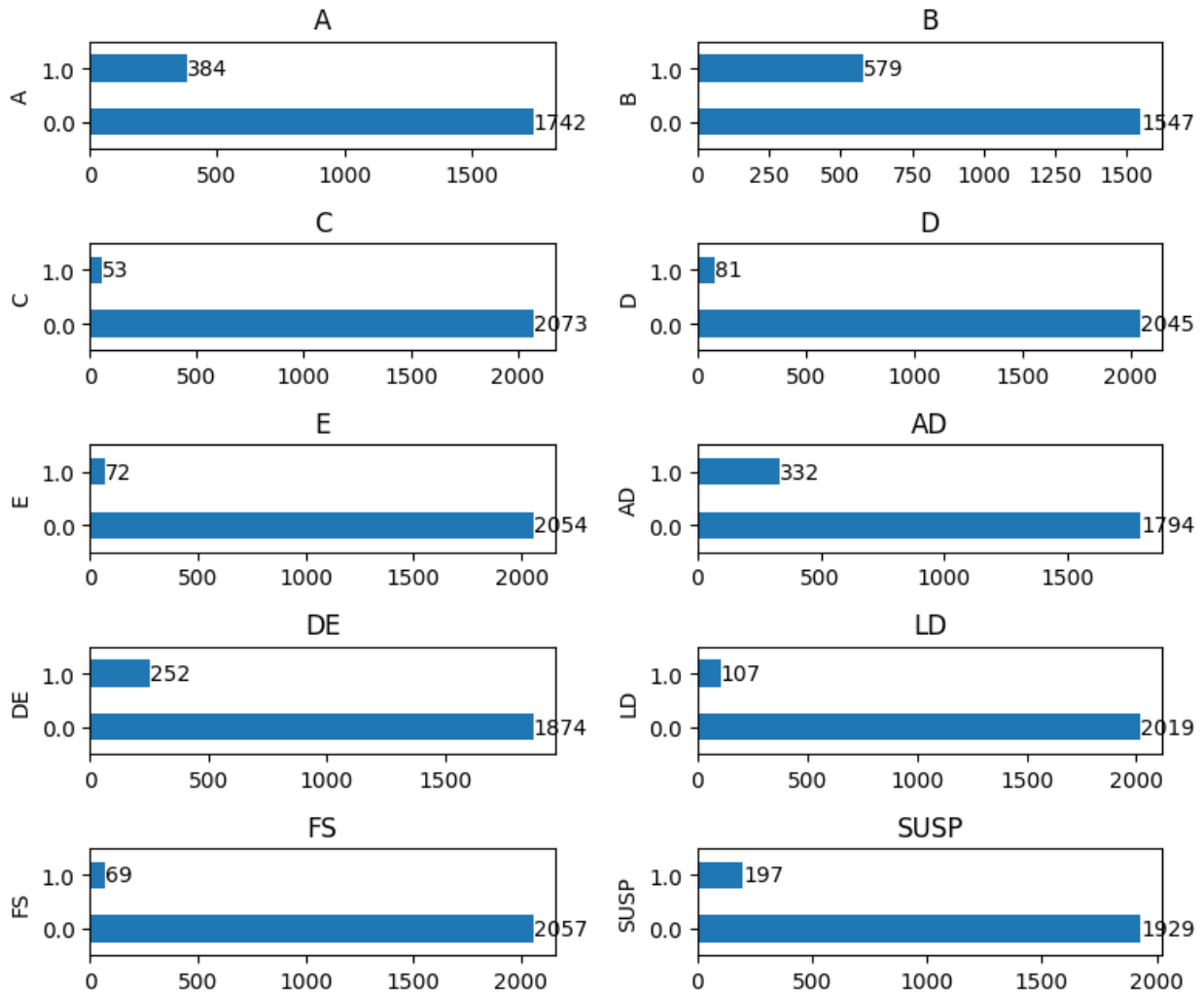
```
lst_num_2 = ['A', 'B', 'C', 'D', 'E', 'AD', 'DE', 'LD', 'FS', 'SUSP']
fig, axes = plt.subplots(nrows=5, ncols=2, figsize=(8,7))

plt.subplots_adjust(
    wspace=1,
    hspace=1
)

for col, ax in zip(df_data[lst_num_2], axes.ravel()):
    df_data[col].value_counts().plot.barh(ax=ax)
    ax.set_title(col)
    ax.bar_label(ax.containers[0])

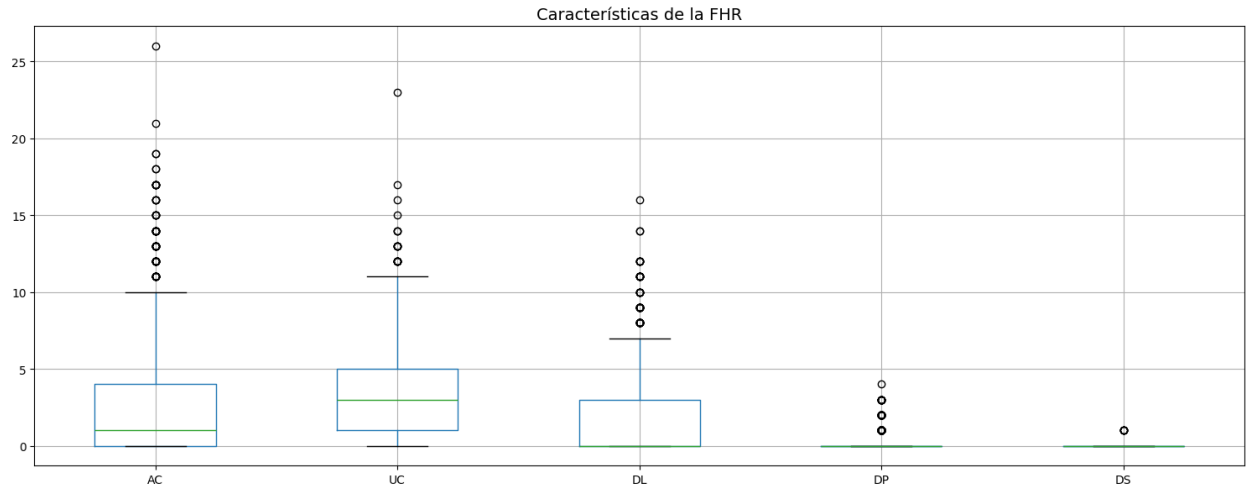
fig.suptitle('CLASS: Clasificación Morfológica', fontsize=12)
plt.tight_layout()
plt.show()
```

CLASS: Clasificación Morfológica



Variables Numéricas

```
lst_num_3 = ['AC', 'UC', 'DL', 'DP', 'DS']  
  
# Se grafican los diagramas de caja de cada una de las variables  
numéricas  
  
boxplot = df_data[lst_num_3].boxplot(figsize=(15,6))  
  
plt.title('Características de la FHR', fontsize=14)  
plt.tight_layout()  
plt.show()
```

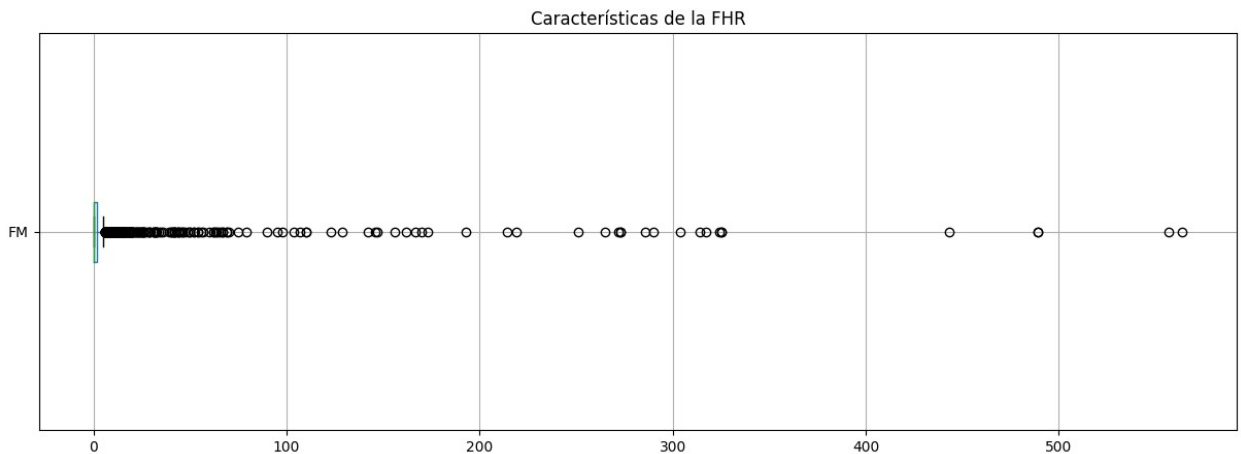


```
lst_num_4 = ['FM']

# Se grafican los diagramas de caja de cada una de las variables
numéricas

boxplot = df_data[lst_num_4].boxplot(figsize=(15,5), vert=False)

plt.title('Características de la FHR', fontsize=12)
plt.show()
```



```
lst_num_5 = lst_num_3 + ['FM']
bins = 10

fig, axes = plt.subplots(3, 2, figsize=(10, 7))
axes = axes.flatten()

for ax, col in zip(axes, lst_num_5):
    data = df_data[col].dropna()

    counts, bins_edges, patches = ax.hist(data, bins=bins)
```

```

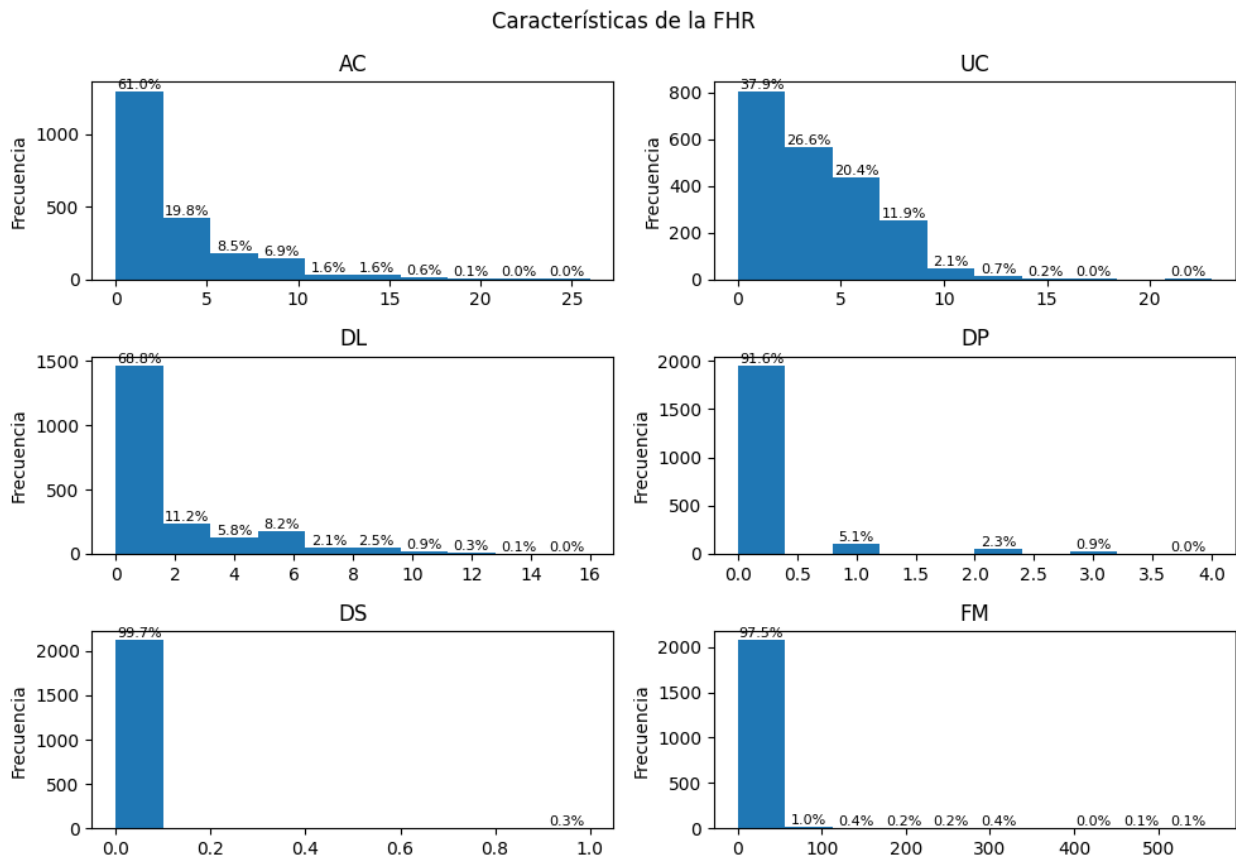
total = len(data)

# Porcentajes sobre cada barra
for count, patch in zip(counts, patches):
    if count > 0:
        percentage = count / total * 100
        ax.text(
            patch.get_x() + patch.get_width() / 2,
            count,
            f'{percentage:.1f}%',
            ha='center',
            va='bottom',
            fontsize=8
        )

ax.set_title(col)
ax.set_ylabel('Frecuencia')

plt.gcf().suptitle('Características de la FHR', fontsize=12)
plt.tight_layout()
plt.show()

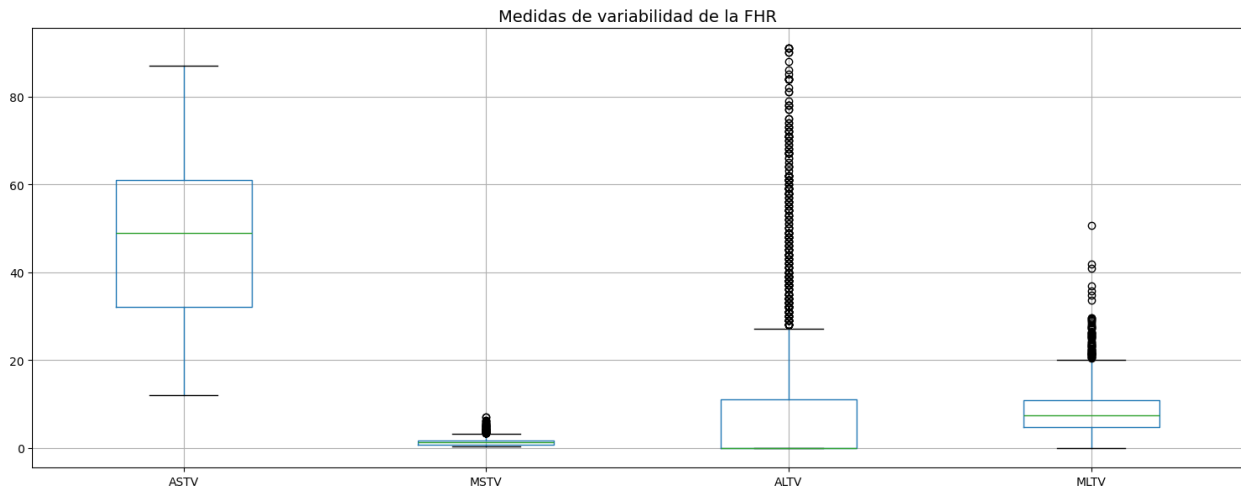
```



```
lst_num_6 = ['ASTV', 'MSTV', 'ALTV', 'MLTV']

# Se grafican los diagramas de caja de cada una de las variables
numéricas

boxplot = df_data[lst_num_6].boxplot(figsize=(15,6))
plt.title('Medidas de variabilidad de la FHR', fontsize=14)
plt.tight_layout()
plt.show()
```



```
bins = 10

fig, axes = plt.subplots(2, 2, figsize=(10, 7))
axes = axes.flatten()

for ax, col in zip(axes, lst_num_6):
    data = df_data[col].dropna()

    counts, bins_edges, patches = ax.hist(data, bins=bins)

    total = len(data)

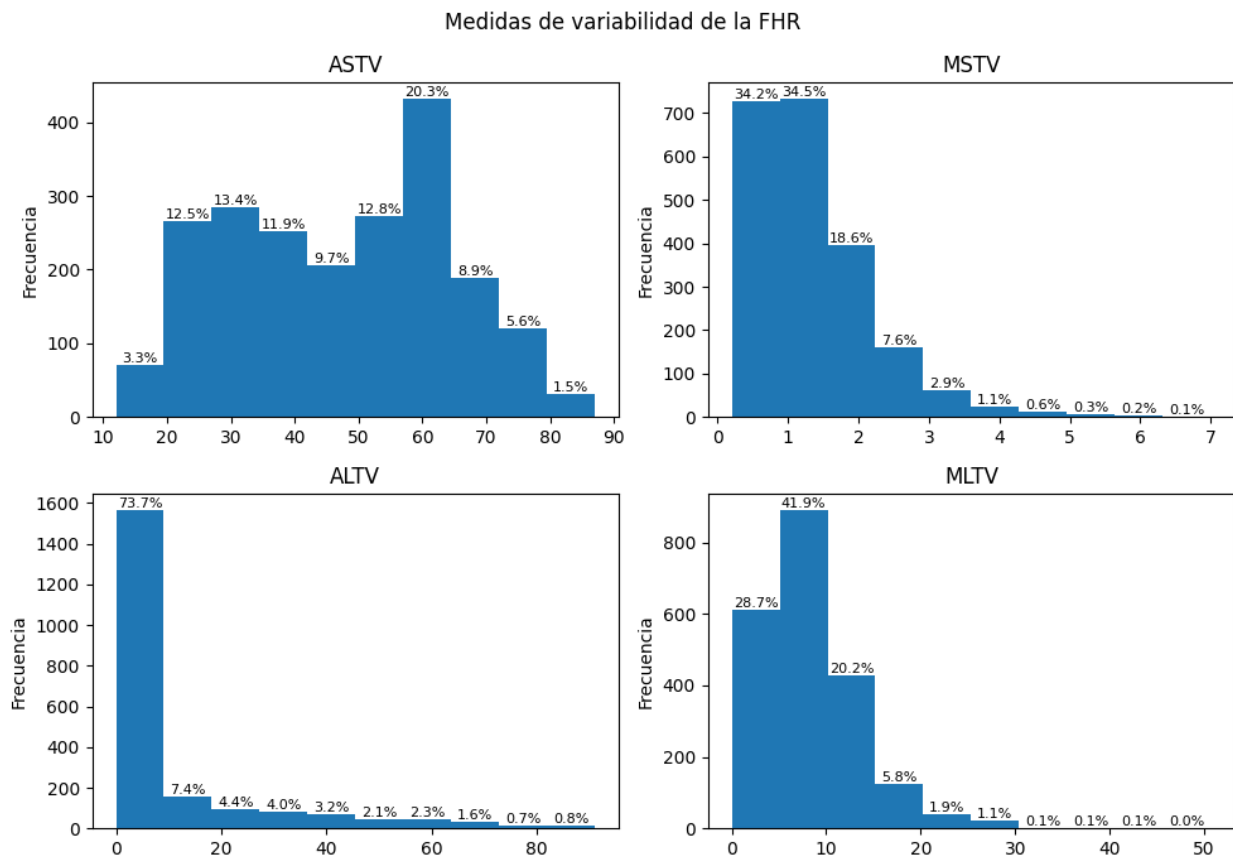
    # Porcentajes sobre cada barra
    for count, patch in zip(counts, patches):
        if count > 0:
            percentage = count / total * 100
            ax.text(
                patch.get_x() + patch.get_width() / 2,
                count,
                f'{percentage:.1f}%',
                ha='center',
                va='bottom',
                fontsize=8
            )
```

```

ax.set_title(col)
ax.set_ylabel('Frecuencia')

plt.gcf().suptitle('Medidas de variabilidad de la FHR', fontsize=12)
plt.tight_layout()
plt.show()

```



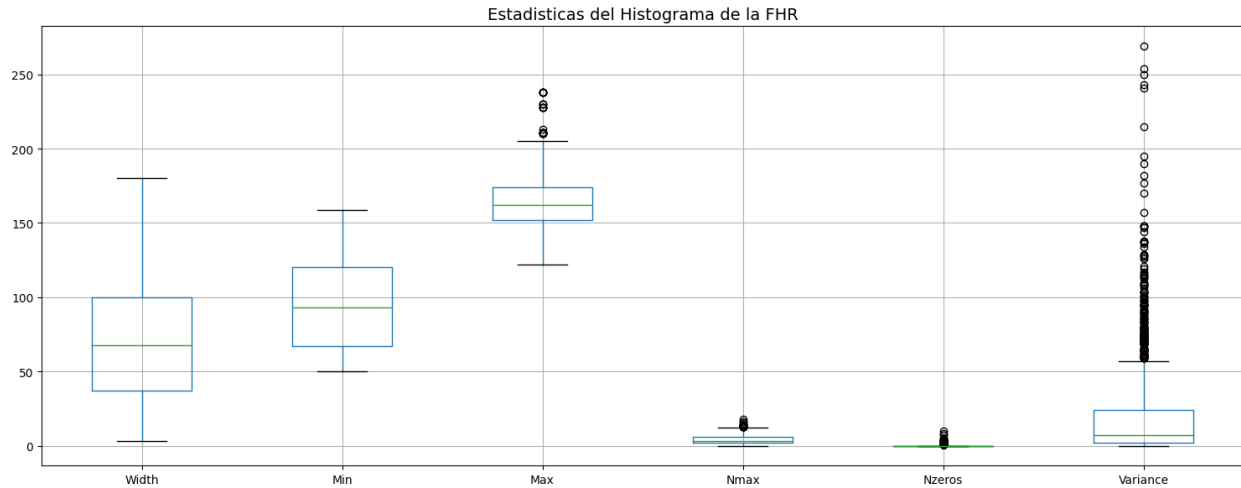
```

lst_num_7 = ['Width', 'Min', 'Max', 'Nmax', 'Nzeros', 'Variance']

# Se grafican los diagramas de caja de cada una de las variables
numéricas

boxplot = df_data[lst_num_7].boxplot(figsize=(15,6))
plt.title('Estadísticas del Histograma de la FHR', fontsize=14)
plt.tight_layout()
plt.show()

```



```
bins = 10

fig, axes = plt.subplots(3, 2, figsize=(10, 7))
axes = axes.flatten()

for ax, col in zip(axes, lst_num_7):
    data = df_data[col].dropna()

    counts, bins_edges, patches = ax.hist(data, bins=bins)

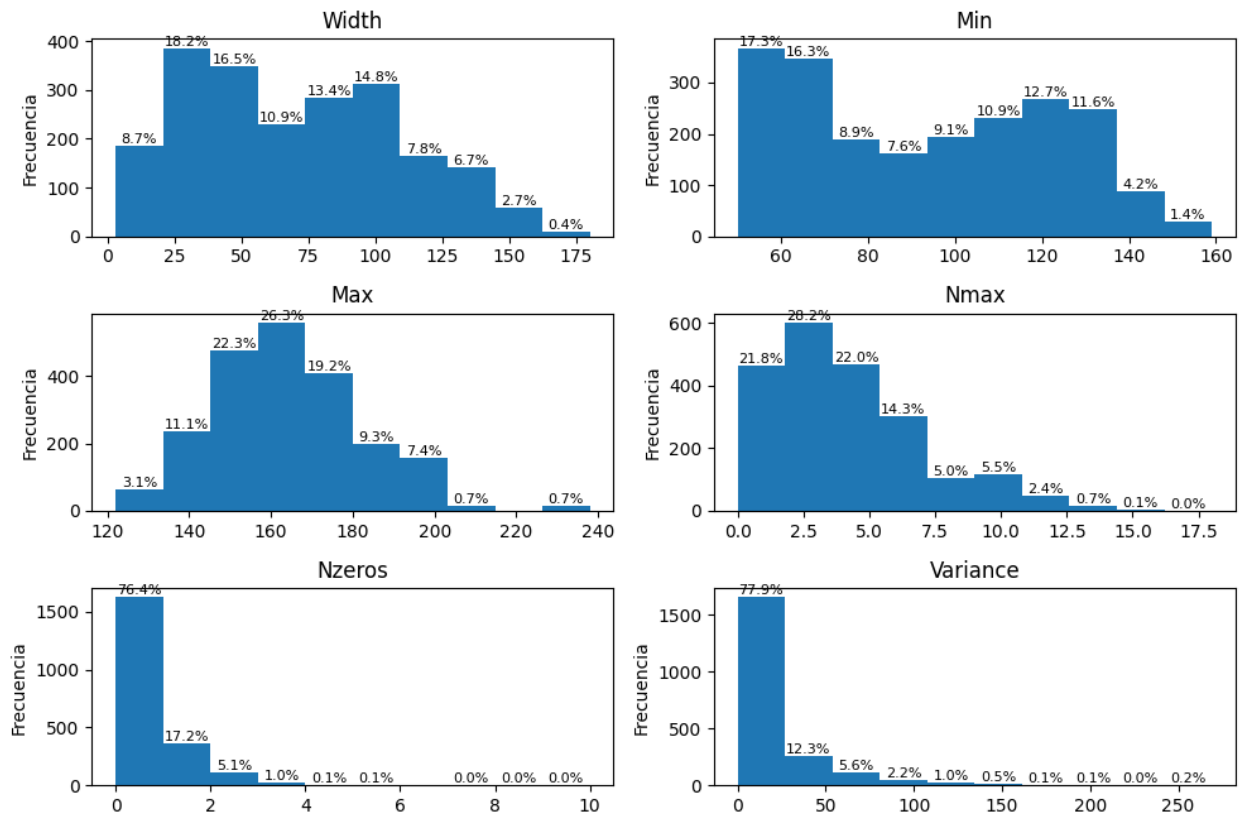
    total = len(data)

    # Porcentajes sobre cada barra
    for count, patch in zip(counts, patches):
        if count > 0:
            percentage = count / total * 100
            ax.text(
                patch.get_x() + patch.get_width() / 2,
                count,
                f'{percentage:.1f}%',
                ha='center',
                va='bottom',
                fontsize=8
            )

    ax.set_title(col)
    ax.set_ylabel('Frecuencia')

plt.gcf().suptitle('Estadísticas del Histograma de la FHR',
    fontsize=12)
plt.tight_layout()
plt.show()
```

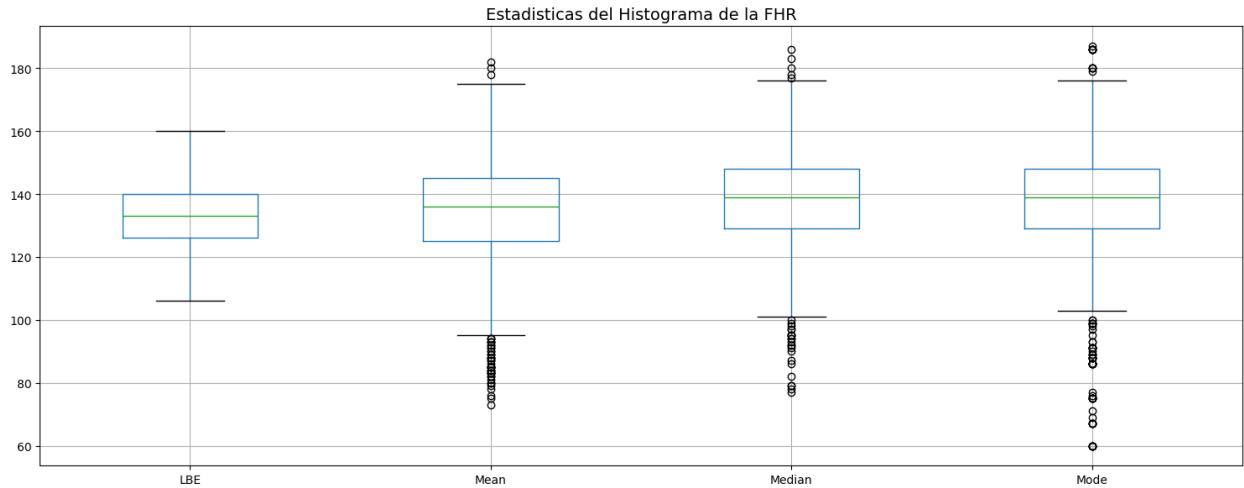
Estadísticas del Histograma de la FHR



```
lst_num_8 = ['LBE', 'Mean', 'Median', 'Mode']

# Se grafican los diagramas de caja de cada una de las variables
numéricas

boxplot = df_data[lst_num_8].boxplot(figsize=(15,6))
plt.title('Estadísticas del Histograma de la FHR', fontsize=14)
plt.tight_layout()
plt.show()
```



```
bins = 10

fig, axes = plt.subplots(2, 2, figsize=(10, 7))
axes = axes.flatten()

for ax, col in zip(axes, lst_num_8):
    data = df_data[col].dropna()

    counts, bins_edges, patches = ax.hist(data, bins=bins)

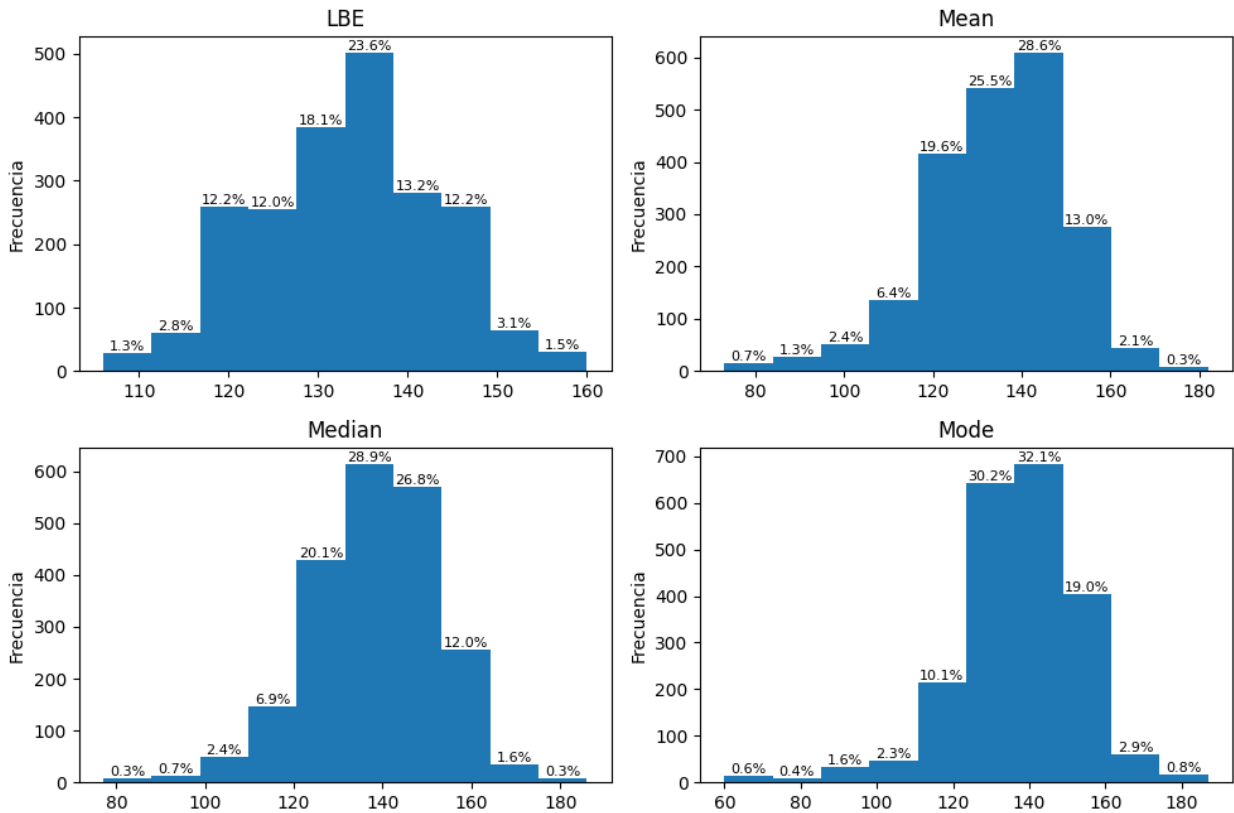
    total = len(data)

    # Porcentajes sobre cada barra
    for count, patch in zip(counts, patches):
        if count > 0:
            percentage = count / total * 100
            ax.text(
                patch.get_x() + patch.get_width() / 2,
                count,
                f'{percentage:.1f}%',
                ha='center',
                va='bottom',
                fontsize=8
            )

    ax.set_title(col)
    ax.set_ylabel('Frecuencia')

plt.gcf().suptitle('Estadísticas del Histograma de la FHR',
    fontsize=12)
plt.tight_layout()
plt.show()
```

Estadísticas del Histograma de la FHR



```
# Se realiza una copia del dataset para modificaciones
df_NSP = df_data.copy()

# Se eliminan las variables target relacionadas con la variable CLASS.

df_NSP.drop(['CLASS', 'A', 'B', 'C', 'D', 'E', 'AD', 'DE', 'LD', 'FS',
'SUSP', 'DS'], axis=1, inplace=True)

# Inicializar el codificador
le = LabelEncoder()

# Transformar la columna 'Tendency' de variable categórica a variable
numérica discreta
df_NSP['Tendency'] = le.fit_transform(df_NSP['Tendency'])
df_NSP['NSP'] = le.fit_transform(df_NSP['NSP'])

df_NSP.sample(10)
```

[illegible]

296	148.0	0.0	18.0	0.0	74.0	0.2	78.0	3.6	0.0	0.0	...
139.0											
329	146.0	0.0	3.0	0.0	77.0	0.2	42.0	4.6	0.0	0.0	...
137.0											
211	125.0	1.0	3.0	0.0	51.0	0.6	0.0	4.9	0.0	0.0	...
125.0											
558	130.0	3.0	0.0	3.0	36.0	2.5	0.0	8.7	2.0	0.0	...
51.0											
2108	136.0	0.0	2.0	6.0	67.0	2.2	0.0	4.4	1.0	0.0	...
100.0											
1149	122.0	4.0	0.0	0.0	26.0	1.4	0.0	5.6	1.0	0.0	...
85.0											
1281	112.0	1.0	0.0	4.0	17.0	2.0	0.0	11.6	2.0	0.0	...
78.0											
421	143.0	0.0	0.0	4.0	65.0	0.6	14.0	13.3	0.0	0.0	...
120.0											

	Max	Nmax	Nzeros	Mode	Mean	Median	Variance	Tendency
NSP								
1629	193.0	6.0	1.0	146.0	129.0	146.0	15.0	1
0								
1307	164.0	2.0	0.0	145.0	139.0	144.0	12.0	2
0								
296	153.0	0.0	0.0	148.0	148.0	149.0	0.0	1
2								
329	149.0	1.0	0.0	142.0	142.0	143.0	0.0	1
1								
211	141.0	1.0	0.0	133.0	132.0	134.0	0.0	1
0								
558	195.0	7.0	2.0	133.0	134.0	134.0	38.0	1
0								
2108	145.0	3.0	0.0	133.0	131.0	136.0	2.0	2
0								
1149	146.0	5.0	0.0	140.0	129.0	134.0	28.0	2
0								
1281	144.0	3.0	1.0	112.0	109.0	111.0	9.0	1
0								
421	158.0	5.0	0.0	145.0	145.0	146.0	2.0	1
1								

[10 rows x 21 columns]

2. AGRUPAMIENTO POR CLUSTERS

```
# Se realiza una copia del dataset para modificaciones
df_cluster = df_NSP.copy()
```

```
# Se eliminan las variables no necesarias para el análisis de
anomalías y clusters.
df_cluster.drop(['MSTV', 'MLTV', 'LBE', 'UC', 'FM', 'DL', 'Tendency',
'Mean', 'Mode', 'Nzeros', 'Width', 'Min', 'Nmax', 'Max'], axis=1,
inplace=True)
```

```
df_cluster.sample(10)
```

	AC	ASTV	ALTV	DP	Median	Variance	NSP
1265	0.0	25.0	9.0	0.0	124.0	4.0	0
476	0.0	62.0	8.0	0.0	148.0	4.0	1
93	0.0	68.0	75.0	0.0	151.0	0.0	2
2121	0.0	79.0	25.0	0.0	152.0	2.0	1
648	0.0	72.0	58.0	0.0	124.0	1.0	1
1995	0.0	62.0	0.0	0.0	125.0	30.0	0
584	1.0	42.0	0.0	0.0	117.0	14.0	0
1951	0.0	60.0	0.0	2.0	102.0	42.0	2
625	7.0	46.0	0.0	0.0	139.0	23.0	0
608	7.0	34.0	0.0	0.0	151.0	11.0	0

```
X = df_cluster.drop('NSP', axis=1)
y_true = df_cluster['NSP']
```

```
scaler = RobustScaler() #StandardScaler() RobustScaler()
MinMaxScaler()
X_scaled = scaler.fit_transform(X)
```

```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

```
# Aqui se guardan las metricas calculadas
res_metric_1 = []
res_metric_2 = []
```

```
# Se realizara las iteracciones del algoritmo k-means con estos
valores de k
ks = [2, 3, 4]
```

```
# Preparamos la figura de 2x2 para los mapas de clústeres
fig, axes = plt.subplots(2, 2, figsize=(10, 10))
axes = axes.flatten()
```

```
# Bucle Principal para ejecutar las iteracciones con los valores de k
predefinidos
for i, k in enumerate(ks):

    # Configuración y entrenamiento del modelo k-means, una vez por
    valor de k
    model = KMeans(n_clusters=k, init='k-means++', n_init=10,
random_state=42)
```

```

labels = model.fit_predict(X_scaled)

# Calculo métricas
metric_1 = {
    'K': k,
    'Silhouette': metrics.silhouette_score(X_scaled, labels),
    'Homogeneity': metrics.homogeneity_score(y_true, labels),
    'Completeness': metrics.completeness_score(y_true, labels)
}

metric_2 = {
    'K': k,
    'V-Measure': metrics.v_measure_score(y_true, labels),
    'ARI': metrics.adjusted_rand_score(y_true, labels),
    'AMI': metrics.adjusted_mutual_info_score(y_true, labels),
    'Calinski': metrics.calinski_harabasz_score(X_scaled, labels)
}

res_metric_1.append(metric_1)
res_metric_2.append(metric_2)

# Distribución de registros
counts = pd.Series(labels).value_counts().sort_index()
percentages =
pd.Series(labels).value_counts(normalize=True).sort_index() * 100

print(f'K-means con K = {k}')
print("\n--- Distribución de Registros ---\n")
for j in range(k):
    print(f"Clúster {j}: {counts[j]} registros
({percentages[j]:.2f}%)")

# Composición Médica (Crosstab)
tabla_comparativa = pd.crosstab(index=labels,
                                columns=y_true,
                                rownames=['Clúster'],
                                colnames=['NSP'])

print("\n--- Composición Médica (NSP) ---")
print(tabla_comparativa)

# Se muestra la pureza en porcentaje por clúster
print("\n--- Pureza por Clúster (%) ---")
print((tabla_comparativa.div(tabla_comparativa.sum(axis=1),
axis=0) * 100).round(2))
print("\n")

# Se grafica PCA
sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=labels,
                palette='viridis', alpha=0.6, ax=axes[i],

```

```

legend='full')
    axes[i].set_title(f'K-means con K = {k}', fontsize=12)
    axes[i].set_xlabel('CP1')
    axes[i].set_ylabel('CP2')
    axes[i].legend(title='Cluster')

print('\n')

# Se grafica La realidad médica (NSP)
sns.scatterplot(x=X_pca[:, 0], y=X_pca[:, 1], hue=y_true.values,
                palette='Set1', alpha=0.6, ax=axes[3], legend='full')
axes[3].set_title('NSP', fontsize=12, fontweight='bold')
axes[3].set_xlabel('CP1')
axes[3].set_ylabel('CP2')

print('\n')
# Se convierte el DataFrame para comparar
df_metric_1 = pd.DataFrame(res_metric_1)
df_metric_2 = pd.DataFrame(res_metric_2)
df_metrics = pd.merge(df_metric_1,
                      df_metric_2.drop(columns=['Calinski']), on='K')

# Se grafica la comparación
df_metrics.set_index('K').plot(kind='bar', figsize=(9, 3))
plt.title('Comparación de Métricas')
plt.ylabel('Puntaje (0 a 1)')
plt.xticks(rotation=0)
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show()

print('\n')
print(df_metric_1)
print('\n')
print(df_metric_2)

```

K-means con K = 2

--- Distribución de Registros ---

Clúster 0: 348 registros (16.37%)
 Clúster 1: 1778 registros (83.63%)

--- Composición Médica (NSP) ---

NSP	0	1	2
Clúster			
0	124	167	57
1	1531	128	119

```

--- Pureza por Clúster (%) ---
NSP      0      1      2
Clúster
0         35.63  47.99  16.38
1         86.11   7.20   6.69

```

K-means con K = 3

```

--- Distribución de Registros ---

```

```

Clúster 0: 1564 registros (73.57%)
Clúster 1: 336 registros (15.80%)
Clúster 2: 226 registros (10.63%)

```

```

--- Composición Médica (NSP) ---
NSP      0      1      2
Clúster
0        1411   122   31
1         119   160   57
2         125    13   88

```

```

--- Pureza por Clúster (%) ---
NSP      0      1      2
Clúster
0         90.22   7.80   1.98
1         35.42  47.62  16.96
2         55.31   5.75  38.94

```

K-means con K = 4

```

--- Distribución de Registros ---

```

```

Clúster 0: 1143 registros (53.76%)
Clúster 1: 321 registros (15.10%)
Clúster 2: 452 registros (21.26%)
Clúster 3: 210 registros (9.88%)

```

```

--- Composición Médica (NSP) ---
NSP      0      1      2
Clúster
0         980   130   33
1         112   152   57
2         452    0    0
3         111    13   86

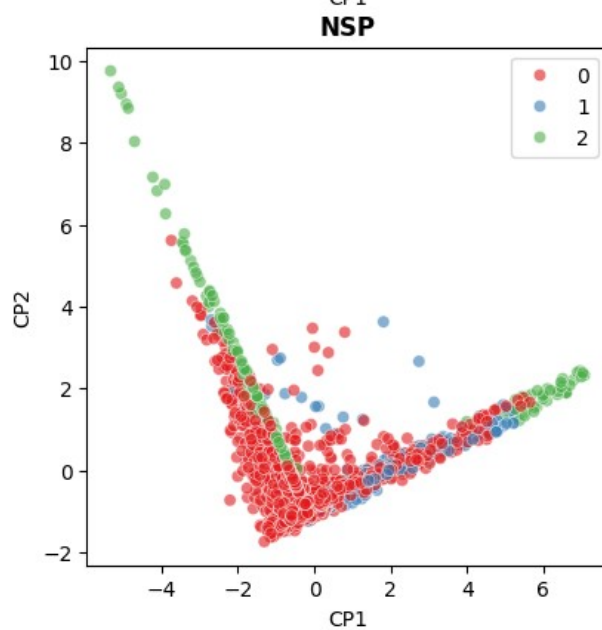
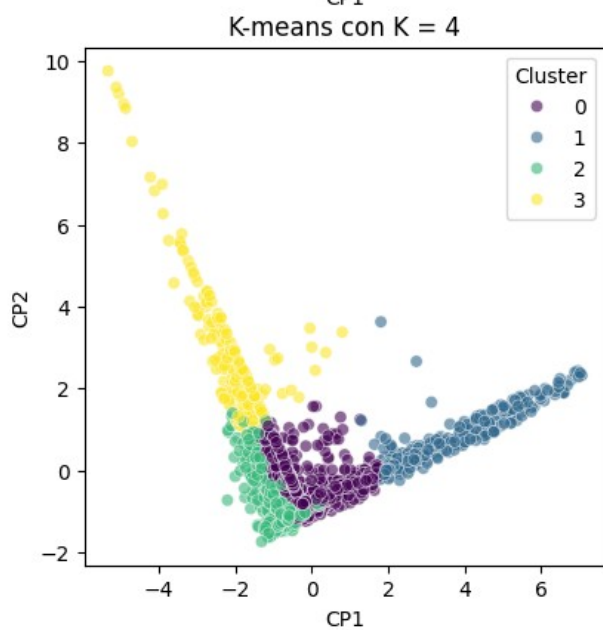
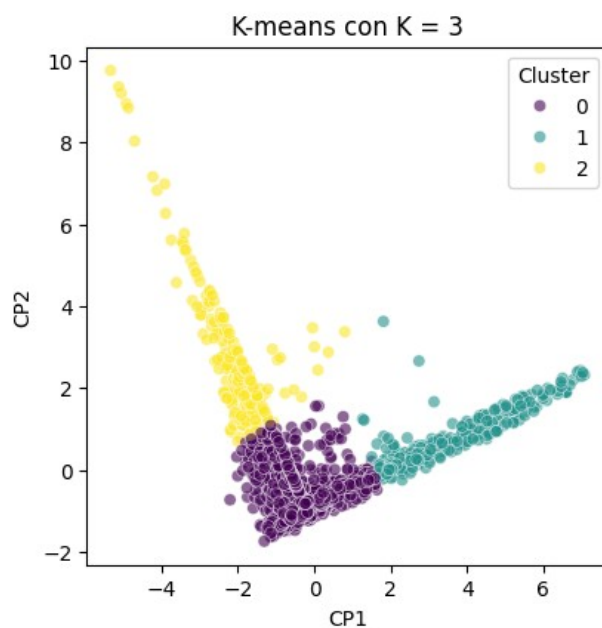
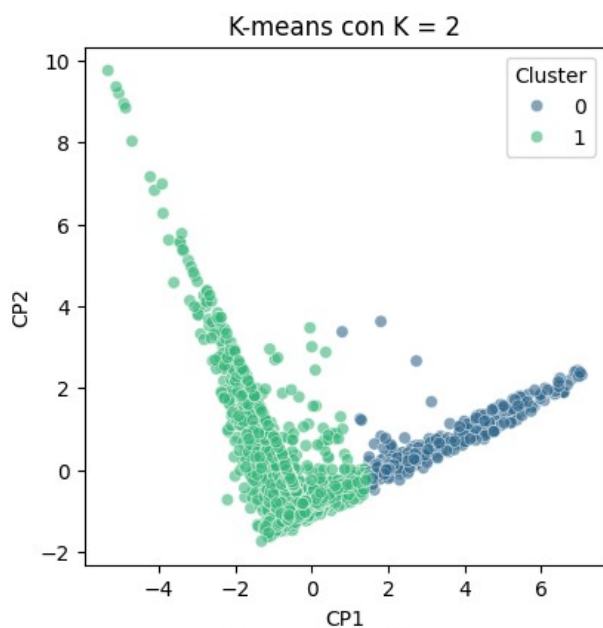
```

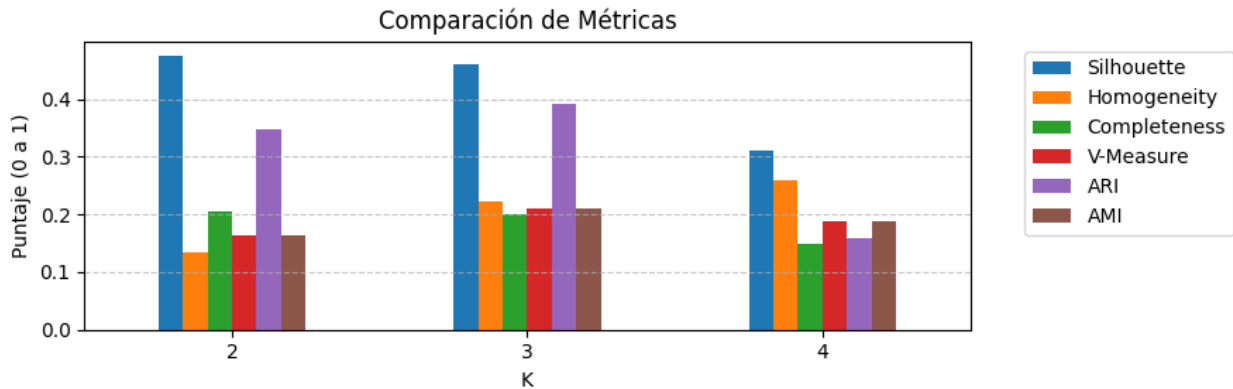
```

--- Pureza por Clúster (%) ---
NSP      0      1      2

```

Clúster			
0	85.74	11.37	2.89
1	34.89	47.35	17.76
2	100.00	0.00	0.00
3	52.86	6.19	40.95





	K	Silhouette	Homogeneity	Completeness
0	2	0.476132	0.135378	0.205091
1	3	0.459798	0.222959	0.199231
2	4	0.311843	0.258744	0.148454

	K	V-Measure	ARI	AMI	Calinski
0	2	0.163098	0.347848	0.162391	1311.983219
1	3	0.210428	0.390583	0.209382	1339.952662
2	4	0.188663	0.159653	0.187418	1251.061550

```

# obtain df information
categories = list(df_metrics)[1:]
values = df_metrics.mean().values.flatten().tolist()
values += values[:1] # repeat the first value to close the circular graph
angles = [n / float(len(categories)) * 2 * pi for n in range(len(categories))]
angles += angles[:1]

# define plot
fig, ax = plt.subplots(nrows=1, ncols=1, figsize=(6, 6),
                        subplot_kw=dict(polar=True))
plt.xticks(angles[:-1], categories, color='black', size=12)
plt.yticks(np.arange(0.5, 1.0, 0.25), ['0.5', '0.75', '#', '1.0', '1.5'],
           color='grey', size=12)
plt.ylim(0, 1)
ax.set_rlabel_position(30)

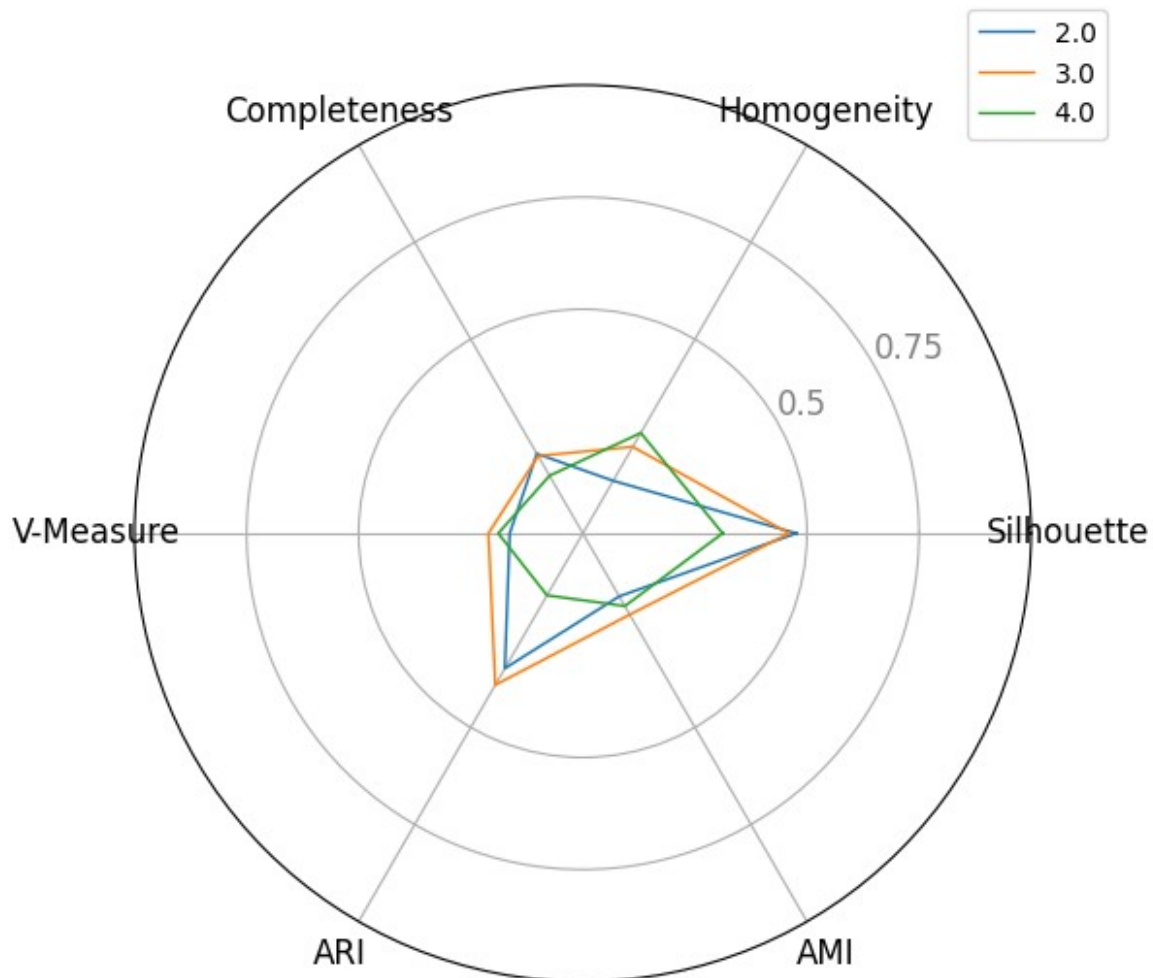
# draw radar-chart:
for i in range(len(df_metrics)):
    val_c1 = df_metrics.loc[i].drop('K').values.flatten().tolist()
    val_c1 += val_c1[:1]
    ax.plot(angles, val_c1, linewidth=1, linestyle='solid',

```

```

label=df_metrics.loc[i]["K"])
# add legend and show plot
plt.legend(bbox_to_anchor=(1.1, 1.1))
plt.show()

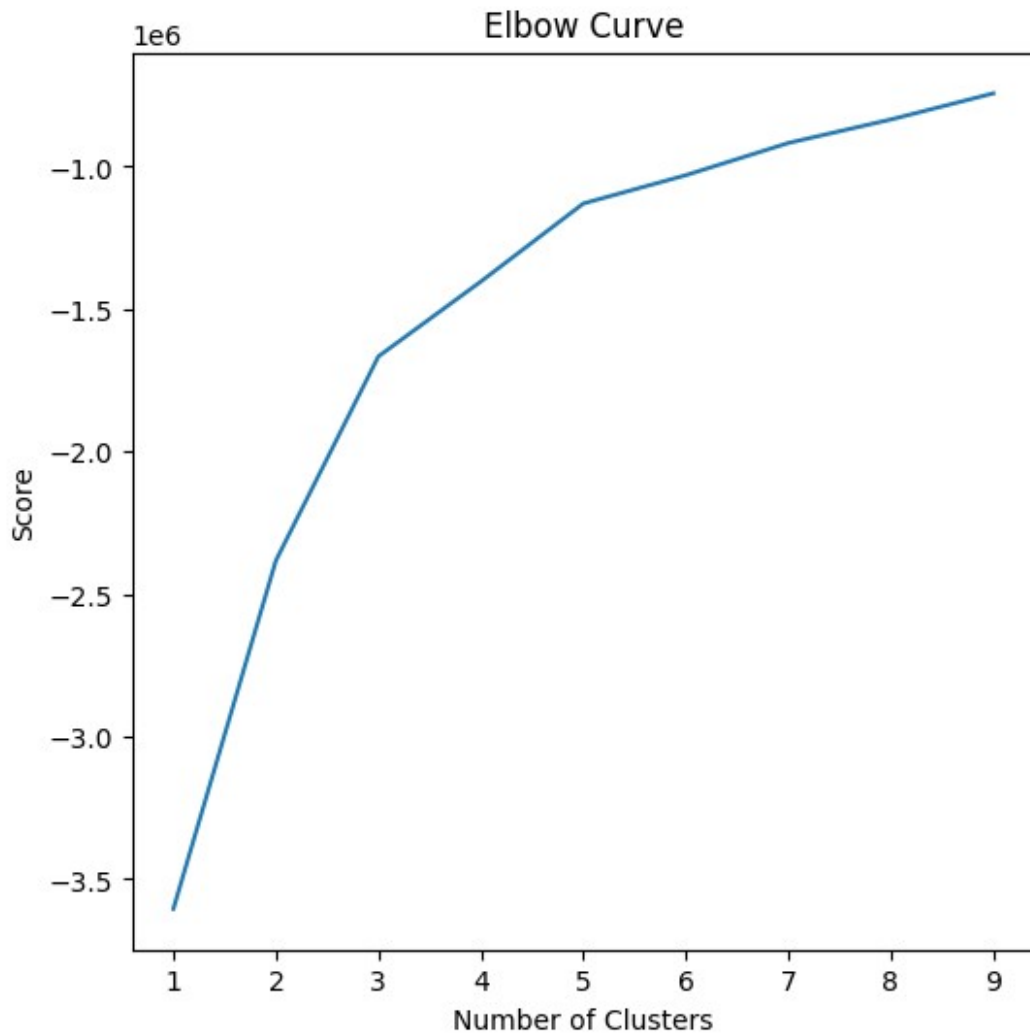
```



```

Nc = range(1, 10)
kmeans = [KMeans(n_clusters=i) for i in Nc]
kmeans
score = [kmeans[i].fit(X).score(X) for i in range(len(kmeans))]
score
plt.rcParams['figure.figsize'] = (6, 6)
plt.plot(Nc, score)
plt.xlabel('Number of Clusters')
plt.ylabel('Score')
plt.title('Elbow Curve')
plt.show()

```



```
def plot_dendrogram(model, **kwargs):
    # Creación de una matriz de vinculación, para graficar el
    # dendrograma
    # Creación de los conteos de las muestras por debajo de cada nodo
    counts = np.zeros(model.children_.shape[0])
    n_samples = len(model.labels_)
    for i, merge in enumerate(model.children_):
        current_count = 0
        for child_idx in merge:
            if child_idx < n_samples:
                current_count += 1 # leaf node
            else:
                current_count += counts[child_idx - n_samples]
        counts[i] = current_count

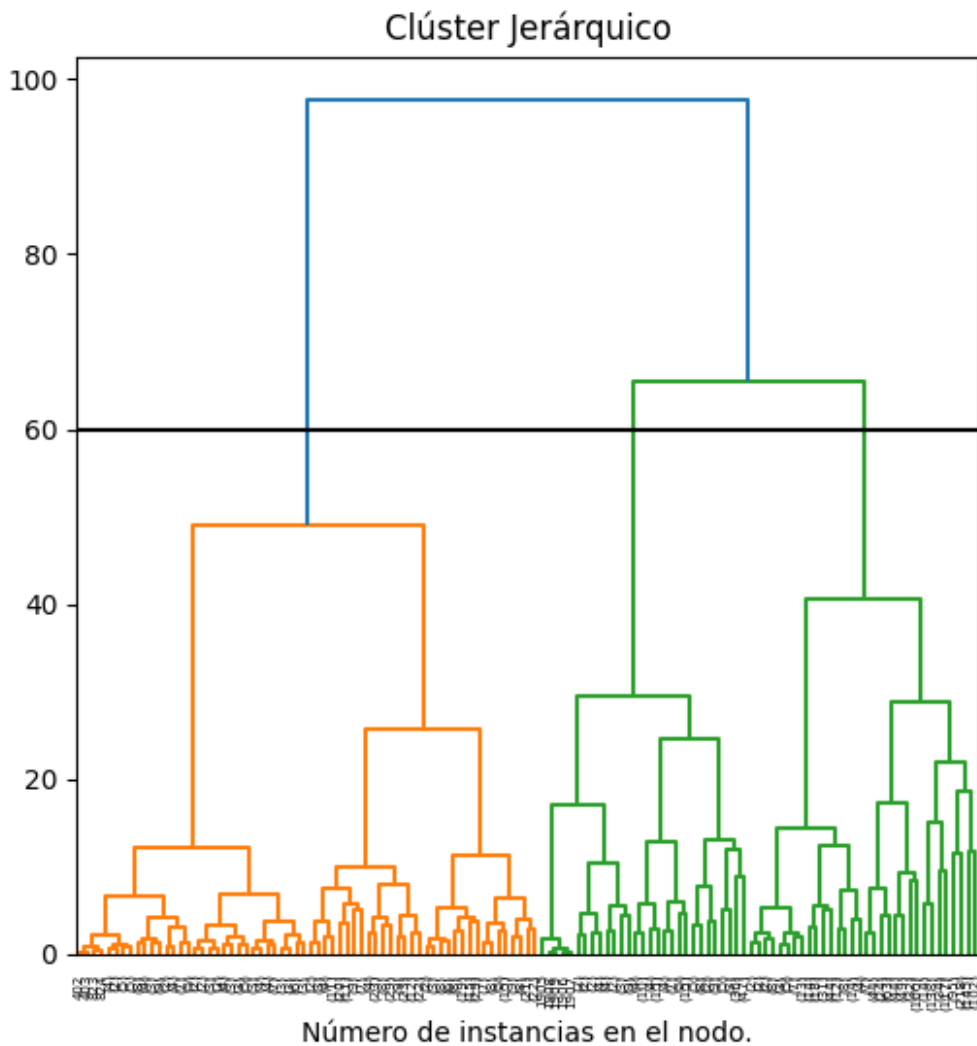
    linkage_matrix = np.column_stack(
        [model.children_, model.distances_, counts]
    ).astype(float)
```

```

# Gráfico correspondiente al Dendrograma
dendrogram(linkage_matrix, **kwargs)

model = AgglomerativeClustering(distance_threshold=0, n_clusters=None)
model = model.fit(X_scaled)
plt.title("Clúster Jerárquico")
plt.rcParams['figure.figsize'] = (15,10)
# Gráfico de niveles del dendrograma
plot_dendrogram(model, truncate_mode="level", p=6)
plt.xlabel("Número de instancias en el nodo.")
# Corte del dendrograma en el número max_d
plt.axhline(y=60, c='k')
plt.show()

```



3. DETECCION DE ANOMALÍAS

Métodos aplicados

1) **Detección a nivel de atributo:** Z-score, Z-score modificado (MAD), IQR y Desviación estándar.

```
# Preparar dataset de trabajo: columnas numéricas y copia
df_anomalia = df_NSP.copy()
numeric_cols =
df_anomalia.select_dtypes(include=[np.number]).columns.tolist()
print(f'Columnas numéricas a usar ({len(numeric_cols)}):',
numeric_cols)

Columnas numéricas a usar (21): ['LBE', 'AC', 'FM', 'UC', 'ASTV',
'MSTV', 'ALTV', 'MLTV', 'DL', 'DP', 'Width', 'Min', 'Max', 'Nmax',
'Nzeros', 'Mode', 'Mean', 'Median', 'Variance', 'Tendency', 'NSP']

# Detección por Z-Score ( $|z| > 3$ )
zscore_outliers = {}
for col in numeric_cols:
    std = df_anomalia[col].std()
    if pd.isna(std) or std == 0:
        zscore_outliers[col] = 0
        continue
    z = (df_anomalia[col] - df_anomalia[col].mean()) / std
    mask = z.abs() > 3
    zscore_outliers[col] = int(mask.sum())

zscore_outliers_s =
pd.Series(zscore_outliers).sort_values(ascending=False)
print('Z-Score clásico (n outliers por atributo):')
display(zscore_outliers_s.head(10))

# Mostrar ejemplos para la variable con más outliers (si existe)
if zscore_outliers_s.iloc[0] > 0:
    top_col = zscore_outliers_s.index[0]
    print(f"Ejemplos de outliers por Z-Score en la variable
'{top_col}':")
    # Recalcular z para la variable seleccionada (top_col) para
asegurar coherencia

    # Opción 1: separar en dos líneas
    z_top = (df_anomalia[top_col] - df_anomalia[top_col].mean()) /
df_anomalia[top_col].std()
    display(df_anomalia.loc[z_top.abs() > 3, top_col].head())

Z-Score clásico (n outliers por atributo):
```

DP	69
ALTV	59

```
Variance    44
Mode        39
AC          35
MSTV        33
MLTV        33
FM          31
DL          28
Nzeros      28
dtype: int64
```

Ejemplos de outliers por Z-Score en la variable 'DP':

```
5      2.0
6      2.0
20     3.0
22     2.0
662    2.0
Name: DP, dtype: float64
```

```
# Detección por Z-Score Modificado (MAD)
modified_z_outliers = {}
for col in numeric_cols:
    x = df_anomalia[col].fillna(0)
    median = x.median()
    mad = (np.abs(x - median)).median()
    if mad == 0:
        modified_z_outliers[col] = 0
        continue
    modified_z = 0.6745 * (x - median) / mad
    mask = modified_z.abs() > 3.5
    modified_z_outliers[col] = int(mask.sum())

modified_z_outliers_s =
pd.Series(modified_z_outliers).sort_values(ascending=False)
print('Z-Score modificado (n outliers por atributo):')
display(modified_z_outliers_s.head(10))
```

Z-Score modificado (n outliers por atributo):

```
Variance    323
AC          305
MSTV        41
MLTV        39
Mode        32
Mean        17
Max         14
Nmax         9
UC           7
Median       7
dtype: int64
```

```

# Detección por Rango Intercuartílico (IQR)
iqr_outliers = {}
for col in numeric_cols:
    q1 = df_anomalia[col].quantile(0.25)
    q3 = df_anomalia[col].quantile(0.75)
    iqr = q3 - q1
    if pd.isna(iqr) or iqr == 0:
        iqr_outliers[col] = 0
        continue
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
    mask = (df_anomalia[col] < lower) | (df_anomalia[col] > upper)
    iqr_outliers[col] = int(mask.sum())

iqr_outliers_s = pd.Series(iqr_outliers).sort_values(ascending=False)
print('IQR (n outliers por atributo):')
display(iqr_outliers_s.head(10))

```

IQR (n outliers por atributo):

FM	310
ALTV	309
Variance	184
AC	83
DL	81
Mode	73
MLTV	71
MSTV	70
Mean	45
Median	28

dtype: int64

```

# Detección por Desviación Estándar (mean ± 3*std)
numeric_cols =
df_anomalia.select_dtypes(include=[np.number]).columns.tolist()
std_outliers = {}
for col in numeric_cols:
    mean = df_anomalia[col].mean()
    std = df_anomalia[col].std()
    if pd.isna(std) or std == 0:
        std_outliers[col] = 0
        continue
    mask = (df_anomalia[col] < (mean - 3*std)) | (df_anomalia[col] >
(mean + 3*std))
    std_outliers[col] = int(mask.sum())

std_outliers_s = pd.Series(std_outliers).sort_values(ascending=False)
print('Desviación estándar (n outliers por atributo):')
display(std_outliers_s.head())

```

Desviación estándar (n outliers por atributo):

```
DP          69
ALTV        59
Variance    44
Mode        39
AC          35
dtype: int64
```

2) Detección a nivel de instancia: Isolation Forest (parámetro `contamination`), Local Outlier Factor y One-Class SVM (exploración de `nu` y `kernel`).

```
# Isolation Forest – uso del parámetro `contamination`
# Asegurar X_scaled
if 'X_scaled' not in globals():
    numeric_cols =
df_anomalia.select_dtypes(include=[np.number]).columns.tolist()
    X = df_anomalia[numeric_cols].fillna(0).values
    scaler = RobustScaler()
    X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scaled)

contamination = 'auto' # Ajustable por el usuario
iso = IsolationForest(n_estimators=200, contamination=contamination,
random_state=42)
iso_preds = iso.fit_predict(X_scaled) # -1 = outlier

df_anomalia['iso_outlier'] = iso_preds == -1
print(f'IsolationForest (contamination={contamination}) – outliers
detectados =', df_anomalia['iso_outlier'].sum())

# Mostrar algunas instancias etiquetadas como outliers por
IsolationForest
display(df_anomalia[df_anomalia['iso_outlier']].head())
```

IsolationForest (contamination=auto) – outliers detectados = 268

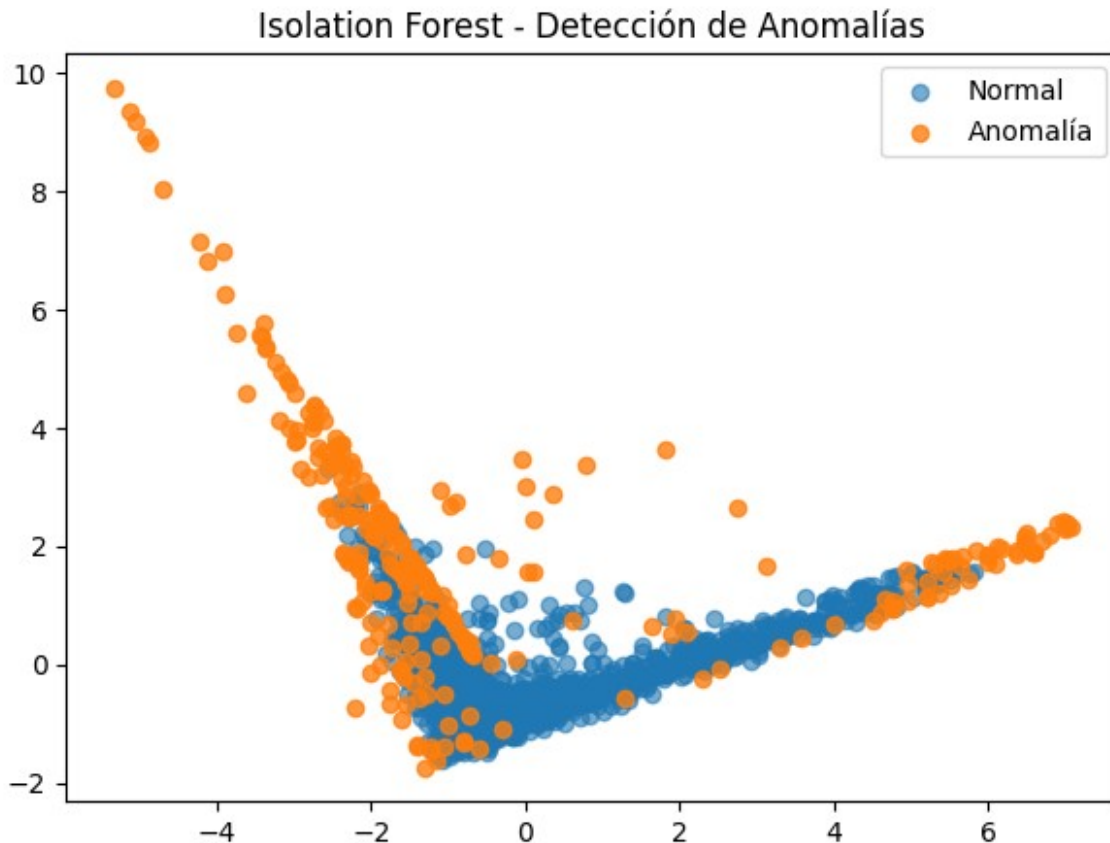
	LBE	AC	FM	UC	ASTV	MSTV	ALTV	MLTV	DL	DP	...
Max \											
0	120.0	0.0	0.0	0.0	73.0	0.5	43.0	2.4	0.0	0.0	...
126.0											
5	134.0	1.0	0.0	10.0	26.0	5.9	0.0	0.0	9.0	2.0	...
200.0											
6	134.0	1.0	0.0	9.0	29.0	6.3	0.0	0.0	6.0	2.0	...
200.0											
14	130.0	7.0	489.0	5.0	21.0	2.3	0.0	7.9	6.0	1.0	...
174.0											
20	129.0	0.0	304.0	4.0	30.0	2.1	0.0	8.5	2.0	3.0	...

182.0

	Nmax	Nzeros	Mode	Mean	Median	Variance	Tendency	NSP
iso_outlier								
0	2.0	0.0	120.0	137.0	121.0	73.0	2	1
True								
5	5.0	3.0	76.0	107.0	107.0	170.0	1	2
True								
6	6.0	3.0	71.0	107.0	106.0	215.0	1	2
True								
14	7.0	0.0	143.0	125.0	135.0	76.0	1	0
True								
20	13.0	0.0	129.0	104.0	120.0	138.0	1	2
True								

[5 rows x 22 columns]

```
# -----  
# Gráfica  
# -----  
plt.figure(figsize=(7, 5))  
plt.scatter(  
    X_pca[iso_preds == 1, 0],  
    X_pca[iso_preds == 1, 1],  
    alpha=0.6,  
    label='Normal'  
)  
plt.scatter(  
    X_pca[iso_preds == -1, 0],  
    X_pca[iso_preds == -1, 1],  
    alpha=0.8,  
    label='Anomalía'  
)  
plt.title("Isolation Forest - Detección de Anomalías")  
plt.legend()  
plt.show()
```

[illegible]

```

6  134.0  1.0  0.0  9.0  29.0  6.3  0.0  0.0  6.0  2.0  ...  6.0
3.0
7  122.0  0.0  0.0  0.0  83.0  0.5  6.0  15.6  0.0  0.0  ...  0.0
0.0
8  122.0  0.0  0.0  1.0  84.0  0.5  5.0  13.6  0.0  0.0  ...  0.0
0.0
9  122.0  0.0  0.0  3.0  86.0  0.3  6.0  10.6  0.0  0.0  ...  1.0
0.0

```

```

      Mode   Mean   Median  Variance  Tendency  NSP  iso_outlier
lof_outlier
0  120.0  137.0   121.0     73.0         2    1         True
True
6   71.0  107.0   106.0    215.0         1    2         True
True
7  122.0  122.0   123.0     3.0         2    2         False
True
8  122.0  122.0   123.0     3.0         2    2         False
True
9  122.0  122.0   123.0     1.0         2    2         False
True

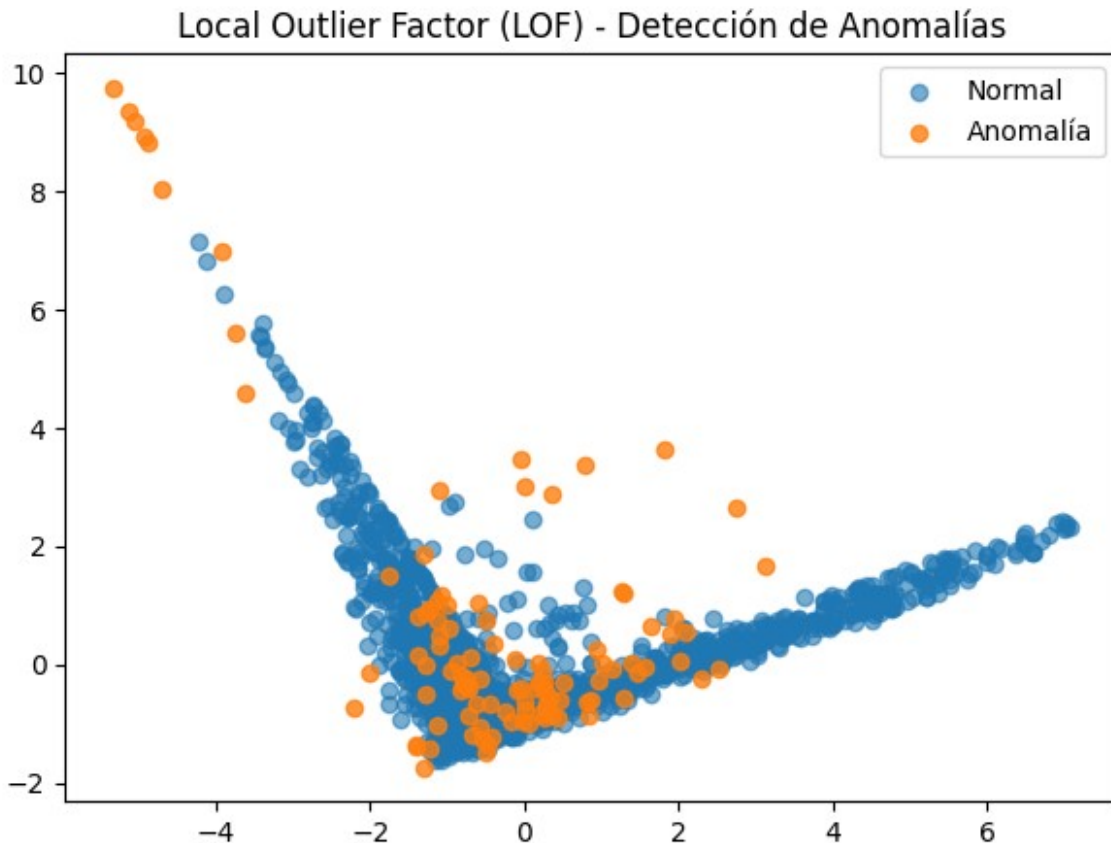
```

```
[5 rows x 23 columns]
```

```

# -----
# Gráfica
# -----
plt.figure(figsize=(7, 5))
plt.scatter(
    X_pca[labels_lof == 1, 0],
    X_pca[labels_lof == 1, 1],
    alpha=0.6,
    label='Normal'
)
plt.scatter(
    X_pca[labels_lof == -1, 0],
    X_pca[labels_lof == -1, 1],
    alpha=0.8,
    label='Anomalía'
)
plt.title("Local Outlier Factor (LOF) - Detección de Anomalías")
plt.legend()
plt.show()

```



```
# One-Class SVM – exploración de `nu` y `kernel`
# Selección de características numéricas y escalado
numeric_cols =
df_anomalia.select_dtypes(include=[np.number]).columns.tolist()
X = df_anomalia[numeric_cols].fillna(0).values
scaler = RobustScaler()
X_scaled = scaler.fit_transform(X)

# Probar combinaciones de nu y kernel y mostrar número de outliers
nus = 0.05
kernels = 'rbf'
results = []
oc = OneClassSVM(nu=nus, kernel=kernels, gamma='scale')
preds = oc.fit_predict(X_scaled)
n_out = (preds == -1).sum()
results.append({'nu': nus, 'kernel': kernels, 'outliers': int(n_out)})

print('One-Class SVM – resultados exploratorios (nu, kernel,
n_outliers):')
print(pd.DataFrame(results))

# Guardar la configuración escogida: nu=0.05, kernel='rbf'
oc_chosen = OneClassSVM(nu=0.05, kernel='rbf', gamma='scale')
```

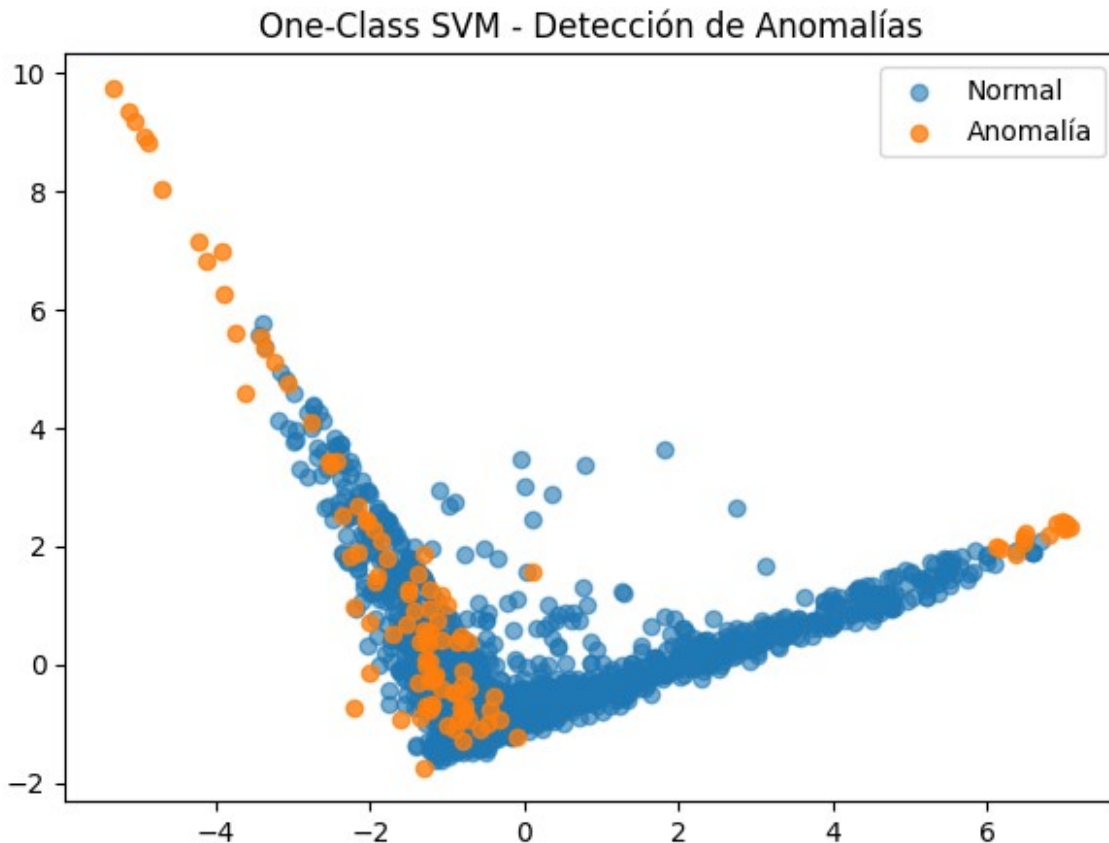
```
df_anomalia['ocsvm_outlier'] = oc_chosen.fit_predict(X_scaled) == -1
print('\nGuardada columna `ocsvm_outlier` con nu=0.05 y kernel="rbf"')
```

One-Class SVM – resultados exploratorios (nu, kernel, n_outliers):

	nu	kernel	outliers
0	0.05	rbf	107

Guardada columna `ocsvm\_outlier` con nu=0.05 y kernel="rbf"

```
# -----
# Gráfica
# -----
plt.figure(figsize=(7, 5))
plt.scatter(
    X_pca[preds == 1, 0],
    X_pca[preds == 1, 1],
    alpha=0.6,
    label='Normal'
)
plt.scatter(
    X_pca[preds == -1, 0],
    X_pca[preds == -1, 1],
    alpha=0.8,
    label='Anomalía'
)
plt.title("One-Class SVM - Detección de Anomalías")
plt.legend()
plt.show()
```



```

### Resumen e intersección de resultados de detección de anomalías
# Mostrar resumen de outliers detectados por cada método y la
intersección entre métodos
cols_instance = ['iso_outlier', 'lof_outlier', 'ocsvm_outlier']
if all(c in df_anomalia.columns for c in cols_instance):
    resumen_instancias = pd.DataFrame({
        'iso': df_anomalia['iso_outlier'].sum(),
        'lof': df_anomalia['lof_outlier'].sum(),
        'ocsvm': df_anomalia['ocsvm_outlier'].sum()
    }, index=[0]).T
    print('Resumen (número de outliers por método):')
    display(resumen_instancias)

# Intersecciones
inter12 = (df_anomalia['iso_outlier'] &
df_anomalia['lof_outlier']).sum()
inter13 = (df_anomalia['iso_outlier'] &
df_anomalia['ocsvm_outlier']).sum()
inter23 = (df_anomalia['lof_outlier'] &
df_anomalia['ocsvm_outlier']).sum()
print('\nIntersecciones:')
print(f'Isolation & LOF: {inter12}')
print(f'Isolation & OCSVM: {inter13}')

```

```
print(f'LOF & OCSVM: {inter23}')
```

```
else:
```

```
    print('Ejecuta las celdas de instancia (IsolationForest, LOF, One-
```

```
Class SVM) primero.')
```

Resumen (número de outliers por método):

	0
iso	268
lof	107
ocsvm	107

Intersecciones:

Isolation & LOF: 36

Isolation & OCSVM: 55

LOF & OCSVM: 20

4. RESULTADOS